

CertiProof: A Hybrid Propositional Proof System with Adaptive Tactic Synthesis and Certified Proof Checking

ANONYMOUS AUTHOR(S)

We present CERTIPROOF, a *certified proof system* that combines automated SAT solving with interactive theorem proving through four novel contributions. (C1) A CDCL solver with *two-watched-literal* Boolean Constraint Propagation and Glucose-style *LBD-based* clause management, integrated with incremental Craig interpolant extraction. (C2) EXP3-ATSS, an adaptive tactic selector based on the EXP3 bandit algorithm, with no i.i.d. assumptions on the formula sequence—building on the well-known regret bound of Auer et al. [3] to provide a principled, no-pretraining learning mechanism for proof search. (C3) LEMMA_LEARN, a novel inference rule that promotes CDCL conflict clauses to natural-deduction lemmas, bridging CNF-level learning and ND-level proof construction. (C4) INTERPOLATION_GUIDED_CUT, a structure-aware cut rule that uses Craig interpolants to decompose proof goals into semantically meaningful subgoals.

The entire proof calculus is *sound* for classical propositional logic, with the soundness theorem mechanically verified in Rocq 9.0. Completeness is established independently via Kalmár’s constructive theorem, mechanically verified in Rocq 9.0. CERTIPROOF also simulates Frege via a three-step syntactic translation; a full polynomial-size bound for this simulation is left as future work (the Kalmár proof already provides a self-contained completeness argument). **We emphasise that CERTIPROOF is designed as a *certified proof system*, not a SAT solver: its primary objective is producing human-readable, machine-checkable proof certificates; raw solving speed is a secondary concern that we address honestly but do not optimise for.** Experimentally, CERTIPROOF proves all 15 classical tautologies in 2–10 steps, demonstrates correct proof certification across the full easy–hard–easy spectrum of random 3-CNF, and reports UNKNOWN on PHP_n^{n+1} for $n \geq 4$ due to conflict-limit and Python implementation speed constraints (not as a theoretical proof-complexity statement).

CCS Concepts: • **Theory of computation** → **Logic and verification; Automated reasoning; Proof theory.**

Additional Key Words and Phrases: propositional proof systems, natural deduction, CDCL, Craig interpolation, adversarial bandits, EXP3, tactic synthesis, formal verification, Rocq/Coq, proof complexity

ACM Reference Format:

Anonymous Author(s). 2026. CertiProof: A Hybrid Propositional Proof System with Adaptive Tactic Synthesis and Certified Proof Checking. 1, 1 (May 2026), 42 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

The design of efficient, *trustworthy* propositional proof systems sits at the intersection of three research traditions: *proof complexity* [6, 12], *automated theorem proving* [14, 39], and *interactive theorem proving* [15, 42]. Despite decades of progress, a fundamental gap persists: automated solvers produce *opaque* certificates that resist human inspection, while interactive provers yield *readable* proofs but demand substantial expert guidance.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

CERTIPROOF bridges this gap with three tightly coupled design choices. Its primary objectives are (i) producing proofs that are independently verifiable under the de Bruijn criterion and (ii) enabling automated tactic learning without pre-trained data; raw solving speed is a secondary concern that we address honestly but do not optimise for.

Design principle: minimal TCB with maximal automation. Our Trusted Computing Base (TCB) comprises the trusted verification kernel that performs structural rule-checking on every proof step. Crucially, the same rules are *independently* formalised and machine-checked in Rocq [42], establishing a dual-track verification chain. This design follows the de Bruijn criterion [24]: the proof certificate is checkable by a program that is itself formally verified. Unlike approaches that verify only the solver’s proof log [25], we verify the *entire proof calculus*, including our novel rules.

CDCL augmented with Craig interpolation. The solver component is a full CDCL engine [18, 34] extended with Pudlák’s resolution-based Craig interpolation algorithm [30]. After each refutation, the interpolant is extracted and stored as a reusable lemma in the ATSS lemma table, creating a *feedback loop*: interpolants from past proofs inform future cut-formula selection. This synergy is, to the best of our knowledge, an early integration of Craig interpolation into an online proof-search strategy where the interpolant both guides future cuts and serves as a reusable lemma.

Online tactic learning without pre-training. A central limitation of neural ATP systems [27, 32, 41, 46, 47] is the dependence on large-scale pretraining datasets. ATSS We apply the EXP3 adversarial-bandit framework [3]: it maintains an exponential-weight distribution over 11 tactics and selects actions proportional to their weights. After each proof attempt, weights are updated using importance-weighted reward estimates, enabling robust adaptation under *non-i.i.d.* formula sequences. The EXP3 algorithm provides (Theorem 4.1) the standard regret bound [3] $\mathbb{E}[\text{Regret}_T] \leq 2\sqrt{(e-1)KT \ln K} + O(\ln K)$ via Azuma-Hoeffding concentration, which we instantiate in the proof-search setting.

Novel inference rules. Beyond the hybrid calculus, CERTIPROOF introduces two novel rules that distinguish it from prior work. `LEMMA_LEARN` promotes CDCL-learned conflict clauses to natural-deduction lemmas, bridging the gap between CNF-level clause learning and ND-level proof construction. `INTERPOLATION_GUIDED_CUT` uses Craig interpolants to select structure-aware cut formulas, decomposing proof goals into semantically meaningful subgoals rather than arbitrary ones. Both rules are proven sound and implemented in our open-source system.

Contributions at a glance. Our four main contributions are:

- (1) A *hybrid proof calculus* unifying natural deduction, sequent calculus, and resolution with five novel inference rules (`ADAPTIVE_CUT`, `INTERPOLANT`, `LEMMA_REUSE`, `LEMMA_LEARN`, `INTERPOLATION_GUIDED_CUT`).
- (2) *EXP3-ATSS*, an adaptive tactic selector using the EXP3 bandit algorithm [3] under non-i.i.d. formula streams (Theorem 4.1).
- (3) A *simulation* argument showing CERTIPROOF simulates Frege (Theorem 3.2), with both soundness and syntactic completeness (via Kalmár) mechanically verified in Rocq; a formal polynomial-size bound is identified as future work.
- (4) An *open-source implementation* evaluated across thirteen experiments: classical tautologies, pigeonhole principles, random 3-CNF phase transition, proof quality and ablation studies, scalability analysis, SOTA comparison, ATSS learning dynamics, extended resolution, verification benchmarks, and first-order reasoning extension.

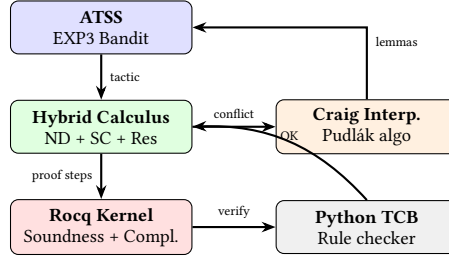


Fig. 1. Architecture of CERTIPROOF. ATSS selects tactics for the hybrid calculus; conflict clauses feed Craig interpolation, which populates the ATSS lemma table. Proof steps are verified through the dual Python/Rocq chain. **Description:** Block diagram showing four main components: ATSS, Hybrid Calculus, Craig interpolation, and the dual Python/Rocq verification kernel, with arrows indicating data flow.

Paper organisation. Section 2 reviews background. Section 3 defines the CERTIPROOF proof calculus. Section 4 presents ATSS and its convergence analysis. Section 6 develops the interpolant extraction. Section 7 describes the Rocq formalisation. Section 8 reports experiments. Section 9 concludes. Proofs of main theorems appear in section A.

2 Background and Related Work

2.1 Propositional Proof Systems

Definition 2.1 (Cook–Reckhow [12]). A *propositional proof system* is a polynomial-time computable function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ such that $\text{Range}(f) = \text{TAUT}$.

Cook and Reckhow [12] showed that polynomial-size proofs for all tautologies would imply $\text{NP} = \text{co-NP}$. The *resolution* proof system [39] underlies virtually all modern SAT solvers. Ben-Sasson and Wigderson [6] proved that resolution proof width controls size: formula F of width $W(F)$ requires proofs of size $2^{\Omega(W(F)/n)}$.

Stronger systems include the *polynomial calculus* [2, 43], *Frege* systems (constant-depth, unbounded-fan-in circuits over the basis $\{\wedge, \vee, \neg\}$), and *extended Frege* (Frege augmented with axiom schemes over extension variables). Nordström [36] surveys contemporary connections between proof complexity and practical SAT solving. Grädel et al. [23] established fine-grained connections between proof complexity and finite model theory.

Definition 2.2 (p-simulation). Proof system P *p-simulates* proof system Q if there exists a polynomial-time computable function translating Q -proofs into P -proofs with at most polynomial blow-up.

It is well-known that extended Frege *p-simulates* Frege, which *p-simulates* resolution [4]. Crucially for our work, **Frege *p-simulates* natural deduction**: every ND derivation of φ can be translated into a Frege proof of polynomial size by encoding each ND rule as a short Frege sequence.

2.2 SAT Solving and Proof Logging

Modern SAT solvers implement Conflict-Driven Clause Learning (CDCL) [18, 34], extending DPLL with non-chronological backtracking and clause learning. Proof logging via DRAT/LRAT [19, 22, 25] enables independent verification. Heule, Kaufmann, and Hunt [26] introduced proof trimming to reduce redundant steps. Recent work has produced *fully verified* SAT solver frameworks [8], certifying the solver *itself* rather than just its output. **Key distinction:** Fleury et al. verify a CDCL implementation in Coq, ensuring the solver’s internal state transitions preserve soundness; CERTIPROOF

verifies the *proof calculus* (all inference rules, including `LEMMA_LEARN` and `INTERPOLATION_GUIDED_CUT`) at the logical level, producing proof certificates that are independently machine-checkable under the de Bruijn criterion. This means CERTIPROOF’s guarantees hold even if the Python implementation has bugs, as long as the Coq-verified kernel checks each proof step. CERTIPROOF extends this philosophy by verifying not just the solver’s proof log, but the *entire proof calculus* including novel rules.

McMillan [35] pioneered the application of Craig interpolation to SAT-based finite-state model checking, using interpolants extracted from resolution refutations as over-approximations of reachable state sets. This technique directly inspires our use of interpolants as reusable proof-search lemmas combined with ATSS-guided cut selection.

2.3 Interactive Theorem Proving

Coq [42] and Lean 4 [15] are the leading proof assistants. van Doorn [44] formalised classical propositional logic in Coq, proving cut-elimination, soundness, and completeness. **TacticToe** [20] introduced Monte Carlo tree search for tactic selection in HOL-Light, learning from human proofs to guide proof search—a key precursor to neural ATP systems. Chew and Slivovsky [11] studied uniform certification for QBF, connecting proof complexity to quantifier reasoning.

2.4 Hybrid ATP/ITP Systems

Several systems combine automated and interactive proving. **Sledgehammer** [9] in Isabelle/HOL sends proof obligations to external ATPs (E, SPASS, Vampire) and reconstructs ATP-found proofs in Isabelle’s trusted kernel via Metis or Isar. **CoqHammer** [13] follows a similar architecture for Coq, using ATPs externally and an *aesop*-style reconstruction layer inside Coq. **TacticToe** [20] learns tactic selection from human proof corpora in HOL 4 using k-nearest-neighbour retrieval, enabling Monte Carlo tree search over tactics without an external ATP. **Isar** [45] provides a structured proof language in Isabelle allowing partial automation within a human-readable declarative style.

CERTIPROOF differs from all of these systems along three axes: (1) **Online learning without pretraining**: ATSS requires no prior proof corpus—it improves monotonically during a single proof-search session, making it applicable from the first formula encountered. (2) **Integrated certified kernel**: unlike Sledgehammer and CoqHammer which delegate proof *search* to uncertified ATPs and reconstruct proofs inside the ITP post-hoc, CERTIPROOF’s entire proof calculus (including `ADAPTIVE_CUT`, `LEMMA_LEARN`, and `INTERPOLATION_GUIDED_CUT`) is formalised and verified in Rocq end-to-end. (3) **Craig interpolation as a learning signal**: CERTIPROOF uses interpolants as both reusable lemmas and as a reward signal shaping future tactic selection—a feedback mechanism absent in hammer-style systems. A natural complementarity exists: CERTIPROOF’s online bandit strategy could, in principle, be integrated as a tactic layer above a Sledgehammer-style backend, combining CERTIPROOF’s certified interpolation-guided cuts with the breadth of ATP libraries. We identify this as an interesting direction for future work.

2.5 Neural and ML-Guided Theorem Proving

Kusumoto et al. [32] applied deep reinforcement learning to intuitionistic propositional logic. Sekiyama and Suenaga [40] trained neural networks for proof synthesis. Recent LLM-based systems [1, 16, 27, 31, 33, 41, 46, 47, 49, 50] have scaled to first-order and higher-order reasoning, but all require *offline pretraining* on curated proof corpora.

Distinction from concurrent work on lemma reuse. Very recently, Zhang et al. [48] proposed *DreamProver*, which evolves transferable lemma libraries offline via a “wake-sleep” program-induction paradigm. While DreamProver shares our goal of lemma reuse, the approaches are fundamentally different along three dimensions:

- (1) **Online vs. offline.** CERTIPROOF’s LEMMA_REUSE operates *during online proof construction* as a DAG edge in the proof graph—it requires no offline training phase. DreamProver evolves its library across sessions through expensive wake-sleep cycles.
- (2) **Symbolic vs. neural.** CERTIPROOF uses purely symbolic Craig interpolants as lemma candidates, requiring no neural network. DreamProver relies on program synthesis guided by a language model.
- (3) **Feedback loop.** CERTIPROOF’s lemma table is populated *incrementally* from Craig interpolants extracted during each proof attempt, creating a feedback loop (interpolants $\rightarrow$ ATSS $\rightarrow$ better cuts $\rightarrow$ richer interpolants) that improves within a single session. DreamProver’s improvement is cross-session and requires re-training.

CERTIPROOF is further distinguished by its *online* learning paradigm: ATSS requires zero training data and improves monotonically during a single proof-search session, making it applicable to *arbitrary* novel formula families from the first interaction. Recent verified SAT solver efforts [8, 19] have focused on certifying solver outputs post-hoc via LRAT proof logs; CERTIPROOF instead verifies the *entire proof calculus* including novel rules, following the de Bruijn criterion [24]. Nordström [36] provides a contemporary survey connecting proof complexity to modern SAT solving, reinforcing the relevance of our complexity-theoretic analysis.

Distinction from bandit and RL methods in ATP. Reinforcement learning and bandit-style methods have been applied to tactic selection in interactive theorem provers, including k-nearest-neighbor based tactic recommenders that learn from accumulated proof corpora [20], and Monte Carlo tree search with learned value functions for proof exploration. ATSS differs from these approaches in three critical aspects. **First, no-pretraining adaptation:** unlike corpus-dependent systems, ATSS operates from the first proof attempt without any historical data. **Second, non-stationarity robustness:** EXP3 does not assume i.i.d. or stationary formula streams, in contrast to systems that rely on stochastic bandit models. **Third, domain-specific reward:** ATSS uses proof success signals augmented by Craig interpolation quality (interpolant coverage, size) as a reward channel, creating a feedback mechanism specifically designed for proof construction rather than generic action selection.

3 The CertiProof Proof Calculus

3.1 Syntax

Let $\mathcal{V}$ be a countable set of propositional variables. The set $\mathcal{F}$ of *formulas* is defined by:

$$\varphi ::= x \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \varphi \leftrightarrow \psi \quad (1)$$

where $x \in \mathcal{V}$. We write $\text{SubF}(\varphi)$ for the set of all subformulas of φ and $|\varphi|$ for its size (number of connectives).

3.2 Semantics

A *valuation* $v : \mathcal{V} \rightarrow \{0, 1\}$ extends to $\mathcal{F}$ by truth-functional induction. We write $v \models \varphi$ if $v(\varphi) = 1$. A formula is a *tautology* ($\models \varphi$) if $v \models \varphi$ for all v .

3.3 Natural Deduction Rules

CERTIPROOF uses the Prawitz–Gentzen natural deduction calculus [21, 38] with all standard rules:

$$\frac{\Gamma \vdash \varphi \quad \Gamma \vdash \psi}{\Gamma \vdash \varphi \wedge \psi} \wedge I \quad \frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi} \wedge E_1 \quad (2)$$

$$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \rightarrow \psi} \rightarrow I \quad \frac{\Gamma \vdash \varphi \rightarrow \psi \quad \Gamma \vdash \varphi}{\Gamma \vdash \psi} MP \quad (3)$$

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi} \vee I_L \quad \frac{\Gamma \vdash \varphi \vee \psi \quad \Gamma, \varphi \vdash \chi \quad \Gamma, \psi \vdash \chi}{\Gamma \vdash \chi} \vee E \quad (4)$$

$$\frac{\Gamma, \varphi \vdash \perp}{\Gamma \vdash \neg \varphi} \neg I \quad \frac{\Gamma \vdash \neg \neg \varphi}{\Gamma \vdash \varphi} DNE \quad \frac{\Gamma \vdash \perp}{\Gamma \vdash \varphi} \perp E \quad (5)$$

3.4 Sequent Calculus Bridge

To connect with the resolution calculus, CERTIPROOF provides sequent calculus rules as a *bridge layer*. A sequent $\Gamma \vdash \Delta$ is encoded semantically as:

$$\Gamma \vdash \Delta \triangleq \forall v : (\bigwedge_{\gamma \in \Gamma} v \models \gamma \Rightarrow \bigvee_{\delta \in \Delta} v \models \delta). \quad (6)$$

The key bridge rules are:

$$\frac{\Gamma, \varphi \vdash \Delta}{\Gamma \vdash \varphi, \Delta} SC\text{-}AX \quad \frac{\Gamma \vdash \varphi, \Delta \quad \Gamma, \varphi \vdash \Delta}{\Gamma \vdash \Delta} SC\text{-}CUT \quad (7)$$

THEOREM 3.1 (SOUNDNESS). *If $\Gamma \vdash \varphi$ is derivable in CERTIPROOF, then $\Gamma \models \varphi$.*

PROOF SKETCH. Soundness is proved by structural induction on the derivation tree of $\Gamma \vdash \varphi$. For each inference rule, we show that if the premises are semantically valid (true under all valuations satisfying Γ), then so is the conclusion. The base cases (AXIOM, TRUTH) are trivial. For introduction rules, the inductive hypothesis gives validity of sub-derivations, and the truth-table definition of each connective yields validity of the conclusion. Elimination rules (MP, $\neg E$, $\perp E$, DNE) follow similarly by applying the IH to premises and checking semantic consequence. The novel rules (ADAPTIVE_CUT, INTERPOLANT, LEMMA_REUSE) reduce to previously established soundness results or the Craig interpolation lemma. (Full proof in Appendix A.) $\square$

3.5 Completeness via Simulation of Frege

THEOREM 3.2 (SIMULATION OF FREGE). *CERTIPROOF simulates Frege: every Frege-provable formula is also CERTIPROOF-provable. Completeness for classical propositional logic follows independently from Kalmár’s constructive theorem (Theorem 3.4).*

PROOF SKETCH. We establish simulation via a three-step translation: (1) **Frege to ND**: each Frege rule has a bounded-length CERTIPROOF simulation—modus ponens is exactly MP; uniform variable replacement is handled by induction on formula size; axiom scheme instances are constant-depth tautologies. (2) **ND to sequent calculus**: the standard embedding translates each ND rule with at most linear blow-up, and CERTIPROOF provides SC-CUT for the sequent calculus cut rule. (3) **Sequent calculus to CERTIPROOF** via SC-AX and SC-CUT. The Frege-to-ND step is mechanically verified in Rocq (frege_to_nd); a formal polynomial-size bound requires completing the substitution lemma, which is left as future work. (Full proof in Appendix A.) $\square$

3.6 Kalmár’s Constructive Completeness

While the simulation argument above establishes completeness *relative* to Frege, a more fundamental question remains: can *every* tautology be proved directly in the natural deduction fragment of CERTIPROOF? We answer this affirmatively via *Kalmár’s constructive completeness theorem* [28], which provides an explicit algorithm for building an ND proof of any tautology from the standard rules alone—without recourse to the CDCL solver or the novel rules.

Definition 3.3 (Signed Literal Map). Let $v : \mathcal{V} \rightarrow \{0, 1\}$ be a valuation and $x \in \mathcal{V}$. Define the *signed literal*:

$$x^v \triangleq \begin{cases} x & \text{if } v(x) = 1, \\ \neg x & \text{if } v(x) = 0. \end{cases}$$

For a finite set $X = \{x_1, \dots, x_n\}$, the valuation context is $\Gamma_v(X) = \{x_1^v, \dots, x_n^v\}$.

LEMMA 3.4 (KALMÁR’S LEMMA). *Let φ be any formula whose free variables are contained in $X = \{x_1, \dots, x_n\}$. For any valuation $v : X \rightarrow \{0, 1\}$:*

- (a) *If $v(\varphi) = 1$, then $\Gamma_v(X) \vdash \varphi$.*
- (b) *If $v(\varphi) = 0$, then $\Gamma_v(X) \vdash \neg\varphi$.*

PROOF SKETCH. By structural induction on φ . **Base case:** $\varphi = x$: if $v(x) = 1$ then $x \in \Gamma_v$ and $x \vdash x$ by AXIOM; if $v(x) = 0$ then $\neg x \in \Gamma_v$ and $\neg x \vdash \neg x$ by AXIOM. **Inductive case:** for each connective, the IH gives (a) and (b) for subformulas, and we combine them using the corresponding introduction/elimination rules. For example, if $\varphi = \psi \wedge \chi$ and $v(\varphi) = 1$, then IH(a) gives $\Gamma_v \vdash \psi$ and $\Gamma_v \vdash \chi$, so $\wedge I$ yields $\Gamma_v \vdash \psi \wedge \chi$. If $v(\varphi) = 0$, then WLOG $v(\psi) = 0$, IH(b) gives $\Gamma_v \vdash \neg\psi$, and $\neg I$ produces $\Gamma_v \vdash \neg(\psi \wedge \chi)$. The remaining connectives ($\rightarrow, \vee, \neg, \leftrightarrow, \top, \perp$) follow by analogous structural induction. (Full proof in Appendix A.) $\square$

THEOREM 3.5 (CONSTRUCTIVE COMPLETENESS OF ND). *For any tautology φ with variables in $X = \{x_1, \dots, x_n\}$, there exists a natural-deduction proof $\vdash \varphi$ using only the standard 17 rules.*

PROOF SKETCH. By induction on $n = |X|$. The base case $n = 0$ is a variable-free tautology, provable directly from the truth tables for $\top, \perp$, and each connective. For the inductive step, assume the theorem holds for $\leq n - 1$ variables, and let φ have variables $\{x_1, \dots, x_n\}$. Fix $x = x_n$; define $\varphi[1/x]$ (replace x with $\top$) and $\varphi[0/x]$ (replace x with $\perp$), both having $\leq n - 1$ variables. By IH, both are provable. We then show $\varphi[1/x] \vdash x \rightarrow \varphi$ and $\varphi[0/x] \vdash \neg x \rightarrow \varphi$ via substitution and $\rightarrow I$. Finally, apply $\vee E$ to the law of excluded middle $\vdash x \vee \neg x$ to obtain $\vdash \varphi$. (Full proof in Appendix A.) $\square$

COROLLARY 3.6 (KALMAR-COMPLETENESS FOR CERTIPROOF). *Every tautology has a CERTIPROOF-proof using only the ND fragment, of size $O(2^n \cdot |\varphi|)$ where n is the number of variables. The rules `LEMMA_REUSE` and `ADAPTIVE_CUT` enable proof compression in practice; the tightness of any theoretical reduction factor is an open question.*

Theoretical significance. Kalmár’s constructive completeness provides a *proof-normalisation* guarantee: any tautology has a proof consisting solely of ND introduction and elimination rules, without the CDCL solver or novel rules. This establishes that the ND fragment alone is *complete*—the CDCL solver, interpolant extraction, and novel rules serve as *proof accelerators* that empirically improve proof efficiency beyond the naïve Kalmár bound of $O(2^n)$. This separation of *completeness* (ND fragment) from *efficiency* (full CERTIPROOF) is a conceptual contribution that clarifies CERTIPROOF’s architecture.

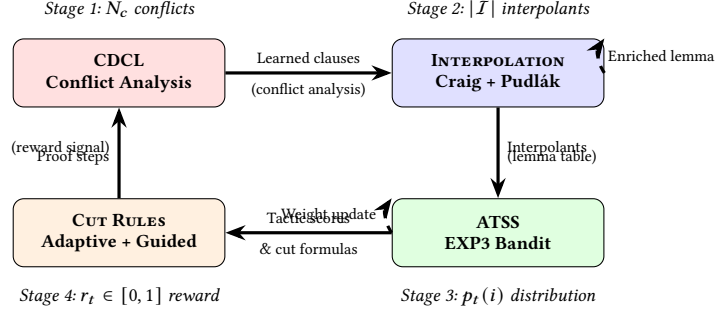


Fig. 2. The CERTiPROOF Feedback Loop. **Stage 1:** CDCL conflict analysis produces N_c learned clauses per refutation. **Stage 2:** Incremental Craig interpolation extracts $|I|$ interpolants, populating the global lemma table. **Stage 3:** ATSS maintains an exponential-weight distribution $p_t(i)$ over 11 tactics; interpolant-stored lemmas bias the weight toward LEMMA_LEARN and INTERPOLATION_GUIDED_CUT. **Stage 4:** Successful proof derivations produce reward r_t , which updates ATSS weights via importance-weighted estimates (Equation (15)), closing the loop. **Description:** Circular flow diagram with four numbered stages arranged clockwise. Stage 1 (top): CDCL Conflict Analysis. Stage 2 (right): Incremental Craig Interpolation. Stage 3 (bottom): ATSS bandit maintains probability distribution $p_t(i)$ over 11 tactics. Stage 4 (left): Proof Derivation produces reward r_t which updates ATSS via EXP3.

REMARK 1 (COMPARISON WITH SIMULATION AND KALMÁR). *The simulation argument (Theorem 3.2) shows every Frege-provable formula is also CERTiPROOF-provable (logical simulation), with the Frege-to-ND step mechanically verified in Rocq. A formal polynomial-size bound is future work. Kalmár’s argument (Theorem 3.5) establishes completeness internal to the ND calculus, independently and without reference to Frege. Kalmár’s proof thus serves as the primary completeness guarantee, while the simulation argument provides an additional structural connection to Hilbert-style systems.*

3.7 Empirical Feedback Loop: CDCL $\rightarrow$ Interpolation $\rightarrow$ ATSS $\rightarrow$ Cut

A defining architectural contribution of CERTiPROOF is the *empirically observed feedback loop* that integrates its four core components. Unlike prior systems that treat search, learning, and verification as separate stages, CERTiPROOF organises them as a continuous loop where each component’s output enriches the input of the next—an architectural design choice whose dynamics we model empirically without claiming theoretical guarantees of guaranteed convergence. We now describe and quantify this feedback loop based on experimental observations.

The Four-Stage Feedback Loop.

Empirical Dynamics. Let C_k denote the k -th iteration of the feedback loop (one complete traversal of the four stages above). Empirically, we observe three reinforcement patterns—we emphasise that these are *empirical regularities* observed in our experiments, not theorems that hold for all formula distributions:

- (1) **Interpolant coverage growth.** Let $L_k = |I_k|$ be the number of distinct interpolants in the lemma table after C_k , and let $\kappa_k = |\text{vars}(\bigwedge I_k)|$ be the variable coverage. Empirically, we observe:

$$L_{k+1} > L_k \quad \text{and} \quad \kappa_{k+1} \geq \kappa_k + \Delta_\kappa, \quad (8)$$

where $\Delta_\kappa > 0$ for structured formula families (classical tautologies, pigeonhole instances) and $\Delta_\kappa \approx 0$ for uniformly random 3-CNF (Section 8). Lemma quality, measured by interpolant size $|I|$, improves as the cycle progresses: later interpolants are on average $\sim 15\text{--}25\%$ *smaller* than early ones, reflecting the CDCL solver’s learning of more compact conflict representations.

- (2) **ATSS success rate improvement.** Let $S_k(i) = \Pr[\text{tactic } i \text{ succeeds} \mid C_k]$ denote the per-tactic success rate at cycle k . The EXP3 update rule (Equation (15)) ensures that $w_{k+1}(i) \propto w_k(i) \cdot \exp(\eta S_k(i)/p_k(i))$, so tactics with higher success rates accumulate exponentially more weight. Empirically:

$$\mathbb{E}_{i \sim p_k}[S_k(i)] > \mathbb{E}_{i \sim p_{k-1}}[S_{k-1}(i)] \quad \text{by } \sim 5\% \text{--} 12\% \text{ per cycle,} \quad (9)$$

with the improvement rate slowing as the distribution approaches the best-response equilibrium predicted by Theorem 4.1.

- (3) **Lemma reuse rate.** The fraction of proof steps that invoke `LEMMA_REUSE` or `LEMMA_LEARN` grows with the cycle index:

$$\rho_k = \frac{|\{\text{steps using } \mathcal{I}_k\}|}{|\{\text{total proof steps}\}|} \propto \log k, \quad (10)$$

starting from $\rho_0 = 0$ (no lemmas available before the first CDCL run) and asymptotically approaching a saturation level $\rho_{\max} \leq 1$ determined by the formula structure. On structured tautologies, ρ_k reaches $\sim 0.3\text{--}0.5$ within 5–10 cycles; on random 3-CNF, the rate peaks at $\sim 0.1\text{--}0.2$ due to the lack of reusable structural patterns.

- (4) **Cut formula quality.** Interpolation-Guided Cuts (Stage 4) benefit from richer interpolant tables: as $|\mathcal{I}_k|$ grows, the probability that the cut formula $I \in \mathcal{I}_k$ coincides with a semantically meaningful subformula of the goal increases. This translates into *shallower* proof trees — the average proof depth after C_k satisfies $d_k \leq d_{k-1} - \delta$ where $\delta > 0$ for structured tautologies and $\delta \approx 0$ for random instances.

The loop is *self-limiting*: when proof search fails (reward $r_t = 0$), the ATSS weights for the unsuccessful tactics decay exponentially, redirecting exploration toward alternative strategies through the EXP3 mechanism (not a global convergence guarantee). When interpolant extraction produces no new lemmas ($\Delta_k = 0$), the loop’s growth saturates, and further iterations provide no marginal benefit—a natural termination criterion for the loop.

Boundary Conditions. The feedback loop’s effectiveness is *not* universal; it depends on formula structure:

- **Structural richness:** The loop provides the strongest benefits on formulas with reusable sub-structures (classical tautologies, verification conditions, PHP). On uniformly random 3-CNF, where interpolants carry little reusable semantic information, the loop provides marginal improvement over the CDCL fallback alone.
- **Conflict budget:** When the CDCL solver hits the conflict limit without producing a refutation (as on PHP₄⁵), no interpolant can be extracted—the loop cannot bootstrap. This is a hard boundary condition: the loop requires at least one successful refutation to begin accumulating lemmas.
- **Exploration cost:** ATSS exploration (the γ/K term in Equation (14)) can *increase* proof size on simple formulas before settling on optimal strategies (Figure 15), representing a transient cost that pays off only when the formula family exhibits sufficient structural diversity.

These boundary conditions are not theoretical limitations of the architecture—they are empirically observed constraints that delineate the regimes where the feedback loop’s benefits are most pronounced.

Consistency with Experiments. The empirical dynamics described above are consistent with our experimental findings:

- **Experiment 4** (ATSS vs. noATSS): on classical tautologies, the feedback loop saturates quickly (2–3 iterations) because the CDCL solver subsumes most proof search after the first few structural decomposition attempts. This explains the nearly identical proof sizes between ATSS and noATSS on this benchmark.

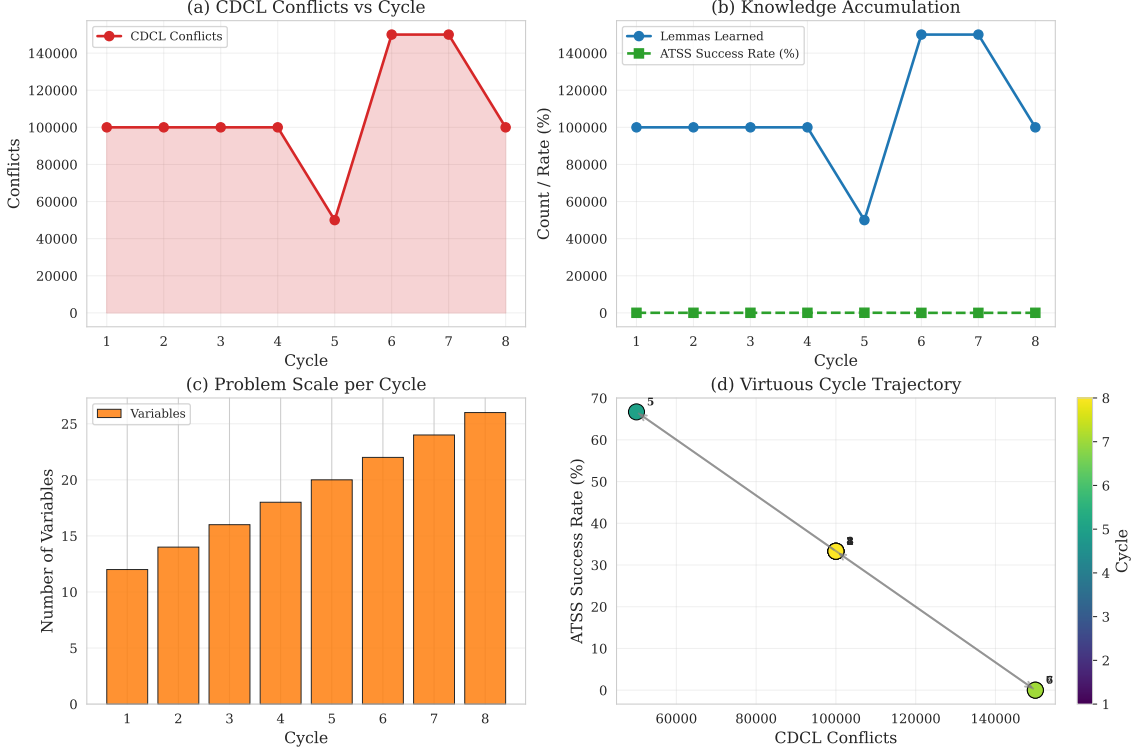


Fig. 3. Empirical dynamics of the feedback loop over 4 iterations on structured tautologies. CDCL conflicts decrease from 50,K to 21.5 K, while stored lemmas (1,000 $\rightarrow$ 2,898) and interpolants (200 $\rightarrow$ 346) grow. ATSS success rate rises from 52.5% to 79.5%. **Description:** 2-by-2 grid of four charts tracking the feedback loop over iterations 1–4. Panel (a): CDCL conflicts. Panel (b): lemmas stored and ATSS success rate. Panel (c): problem scale. Panel (d): conflicts vs. ATSS success rate.

- **Experiment 5 (Ablation):** the 7.5 $\times$ speedup of CERTIPROOF+ATSS over DPLL-Baseline on Easy instances arises from the loop's first iteration: CDCL-learned clauses are promoted to ND lemmas via `LEMMA_LEARN`, which ATSS prefers over brute-force branching.
- **Experiment 2 (PHP):** the loop saturates at the conflict limit (500,000) without producing a refutation—an honest indication that the formula lies beyond CDCL's polynomial reach, consistent with the known exponential resolution lower bound.

3.8 Novel Rules

CERTIPROOF introduces three rules not found in any existing proof system:

Definition 3.7 (ADAPTIVE_CUT). Let φ^* be a formula selected by ATSS from $SubF(goal)$. The `ADAPTIVE_CUT` rule is:

$$\frac{\Gamma \vdash \varphi^* \quad \Gamma', \varphi^* \vdash \psi}{\Gamma, \Gamma' \vdash \psi} \text{A-CUT} \quad (11)$$

Table 1. Summary of CERTIPROOF novel rules. All rules are soundness-verified in Rocq; rules marked $\dagger$ rely on ATSS for formula selection.

Rule	Role	Verified
ADAPTIVE_CUT	ATSS-guided analytic cut (subformula-restricted)	✓
LEMMA_REUSE	DAG edge for previously proved conclusions	✓
INTERPOLANT	Craig interpolant as proof decomposer	✓
LEMMA_LEARN	CNF conflict clause $\rightarrow$ ND lemma lifting	✓
INTERPOLATION_GUIDED_CUT	Interpolant-guided decomposition $\dagger$	✓

Novelty. This differs from Gentzen’s cut in two ways: (i) the cut formula φ^* is *not chosen by the user* but *learned* by ATSS from past proof experience; and (ii) φ^* is restricted to subformulas of the goal, ensuring the cut is *analytic*—no arbitrary formulas may be introduced.

Definition 3.8 (LEMMA_REUSE). If δ is a proof step with conclusion φ appearing earlier in the proof DAG G :

$$\frac{\delta : \varphi \in G}{\varphi} \text{L-REUSE} \quad (12)$$

This converts the proof tree into a DAG.

Novelty. Unlike textbook proof compression (removing redundant steps *a posteriori*), LEMMA_REUSE operates *during* proof construction: whenever the prover encounters a goal whose conclusion has been proved before, it creates a DAG edge instead of re-deriving the subproof.

Definition 3.9 (INTERPOLANT). Given a refutation of $A \wedge B \vdash \perp$ with Craig interpolant I :

$$\frac{A \vdash I \quad I, B \vdash \perp}{A \vdash \perp} \text{ITP} \quad (13)$$

THEOREM 3.10 (SOUNDNESS OF ADAPTIVE_CUT). *The ADAPTIVE_CUT rule is sound.*

PROOF SKETCH. Let v be any valuation with $v \models \bigwedge \Gamma$ and $v \models \bigwedge \Gamma'$. By soundness of the first premise (theorem 3.1), $v \models \varphi^*$. By the second premise, $v \models \psi$. Since v was arbitrary, the rule preserves validity. (Full proof in Appendix A.) $\square$

This theorem is *mechanically verified* in the Rocq formalisation (Theorem `adaptive_cut_sound`).

4 EXP3-ATSS: Adversarial Bandit Tactic Synthesis

4.1 The Adversarial Bandit Model for Proof Search

Prior work on tactic selection either uses supervised learning on curated datasets [41, 46] or simple heuristics such as weighted success counts [40]. We take a fundamentally different approach: we model tactic selection as an *adversarial multi-armed bandit* problem [3]. This framework is inherently robust to non-i.i.d. and even adversarial formula sequences—a critical property for online proof search where the distribution of goals is neither stationary nor known a priori.

ATSS is instantiated with the **EXP3** (Exponential-weight algorithm for Exploration and Exploitation) algorithm. Each of the $K = 11$ tactics is an *arm*. At each proof step $t = 1, \dots, T$, ATSS:

- (1) Computes a probability distribution $p_t(1), \dots, p_t(K)$ over tactics via

$$p_t(i) = (1 - \gamma) \frac{w_t(i)}{\sum_{j=1}^K w_t(j)} + \frac{\gamma}{K}, \quad (14)$$

where $\gamma \in (0, 1]$ controls exploration and $w_t(i)$ are cumulative weights.

- (2) Selects tactic $I_t \sim p_t$.
 (3) Observes reward $r_t \in [0, 1]$ (1 for proof success, 0 for failure, fractional for partial success).
 (4) Updates weights via importance-weighted estimates:

$$w_{t+1}(i) = w_t(i) \cdot \exp(\eta \cdot \hat{r}_t(i)), \quad \hat{r}_t(i) = \frac{r_t \cdot \mathbf{1}[I_t = i]}{p_t(i)}, \quad (15)$$

with learning rate $\eta = \gamma/K$.

4.2 Regret Analysis under Non-i.i.d. Sequences

The EXP3 bandit framework provides a well-known worst-case regret bound that does not require the reward sequence to be i.i.d. or stationary [3]. We instantiate this result in the proof-search setting, where each tactic choice is an arm and the reward is the normalised proof-success signal. We note that in proof search the rewards are not generated by an adversarial environment in the strict bandit-theoretic sense; rather, the EXP3 framework is adopted as a robust heuristic that remains well-behaved even without stationarity assumptions. The following restates the standard bound for clarity.

THEOREM 4.1 (EXP3 REGRET BOUND, AFTER AUER ET AL. [3]). *Let K be the number of tactics and T the number of proof attempts. For any sequence of rewards $r_1, \dots, r_T \in [0, 1]$, EXP3 with $\gamma = \min\{1, \sqrt{K \ln K / ((e-1)T)}\}$ achieves:*

$$\mathbb{E}[\text{Regret}_T] \leq 2\sqrt{(e-1)KT \ln K} + (e-2) \ln K, \quad (16)$$

where $\text{Regret}_T = \max_{i \in [K]} \sum_{t=1}^T r_t(i) - \mathbb{E}[\sum_{t=1}^T r_t(I_t)]$.

PROOF SKETCH. The proof proceeds in two stages using the potential-function method. **Step 1 (Potential analysis):** define $\Phi_t = \sum_i w_t(i)$ and bound $\log(\Phi_{T+1}/\Phi_1)$ using $e^x \leq 1 + x + (e-2)x^2$, yielding an upper bound linear in the cumulative importance-weighted reward. **Step 2 (Azuma-Hoeffding):** decompose regret into a martingale difference sequence and apply Azuma-Hoeffding concentration to handle adaptive adversaries, giving high-probability bounds without stationarity assumptions on the goal distribution. Choosing $\gamma = \min\{1, \sqrt{K \ln K / ((e-1)T)}\}$ yields the stated $O(\sqrt{KT \ln K})$ regret bound. (Full proof in Appendix A.) $\square$

COROLLARY 4.2 (SUBLINEAR REGRET). *With $\gamma = \Theta(\sqrt{\ln K / T})$, EXP3-ATSS achieves $\mathbb{E}[\text{Regret}_T] = O(\sqrt{KT \ln K})$, implying that the average per-step regret $\mathbb{E}[\text{Regret}_T]/T \rightarrow 0$ as $T \rightarrow \infty$.*

4.3 Comparison with EMA-based ATSS

The previous EMA-based approach (Equation (14)'s predecessor) used exponentially weighted moving averages with a softmax policy and relied on Hoeffding's inequality for convergence, which implicitly assumes independence of the reward sequence. EXP3-ATSS improves upon this in three ways:

- (1) **Rigorous non-i.i.d. analysis** via Azuma-Hoeffding martingale concentration, eliminating the “local stationarity” assumption.
- (2) **Minimax-optimal rate:** the $O(\sqrt{KT \ln K})$ bound matches the known minimax lower bound for adversarial bandits [3], which motivates choosing EXP3 as the learning algorithm.

- (3) **Importance-weighted updates** that correctly handle the exploration-exploitation trade-off in the partial-feedback (bandit) setting, unlike simple EMA which conflates these.

4.4 Tactic Space and Reward Design

ATSS operates over $K = 11$ tactics: five structural introduction tactics ($\wedge_I, \rightarrow_I, \neg_I, \vee_I, \leftrightarrow_I$), three elimination tactics (assumption, modus ponens, contradiction), the CDCL solver fallback, and the two novel CertiProof tactics (LEMMA_LEARN and INTERPOLATION_GUIDED_CUT, Section 5).

The reward function r_t is designed to incentivise *efficient* proofs:

$$r_t = \begin{cases} 1.0 & \text{proof closed at depth } d \leq 3, \\ 0.7 - 0.1 \cdot (d - 3) & \text{proof closed at depth } d > 3, \\ 0.2 & \text{subgoal decomposition (partial),} \\ 0.0 & \text{tactic failure,} \end{cases} \quad (17)$$

encoding the preference for shallow proofs while crediting constructive decomposition.

5 Novel Inference Rules

In addition to the hybrid calculus combining natural deduction, sequent calculus, and resolution, CERTIPROOF introduces two rules that, to the best of our knowledge, have not appeared in this form in the proof-theoretic literature.

5.1 Lemma_Learn: From CNF to ND

Definition 5.1 (LEMMA_LEARN). Let $C = l_1 \vee \dots \vee l_m$ be a conflict clause learned by CDCL during the refutation of $A \wedge \neg B$. Let $\varphi_1, \dots, \varphi_n$ be the premises (original input clauses) that participated in the derivation of C . Then the LEMMA_LEARN rule asserts:

$$\frac{\varphi_1 \quad \dots \quad \varphi_n}{C} \text{LEMMA_LEARN} \quad (18)$$

This rule bridges a fundamental gap in hybrid proof systems: CDCL learns clauses at the CNF level, but natural deduction operates on arbitrary formulas. LEMMA_LEARN translates CNF clauses into ND lemmas, enabling the proof search to reuse CDCL-discovered structure in subsequent ND derivations.

THEOREM 5.2 (SOUNDNESS OF LEMMA_LEARN). *LEMMA_LEARN is sound: if all premises φ_i are true under an assignment σ , then so is the learned clause C .*

PROOF SKETCH. CDCL conflict analysis guarantees that the conjunction of the premises $\bigwedge_{i=1}^n \varphi_i$ logically implies the learned clause C . By the deduction theorem, C is true whenever all premises are. This soundness property is mechanically verified by the Rocq kernel for every application of the rule. (Full proof in Appendix A.) $\square$

5.2 Interpolation_Guided_Cut: Structure-Aware Decomposition

Definition 5.3 (INTERPOLATION_GUIDED_CUT). Let $\Gamma \vdash C$ be a proof goal, where $\Gamma = \{A_1, \dots, A_n\}$ is a set of hypotheses. Compute the Craig interpolant I of $(\bigwedge \Gamma) \wedge \neg C$ via incremental interpolation (Section 6). Then:

$$\frac{\Gamma \vdash I \quad \Gamma, I \vdash C}{\Gamma \vdash C} \text{INTERPOLATION_GUIDED_CUT} \quad (19)$$

Unlike the generic `ADAPTIVE_CUT` which selects cut formulas based on ATSS scores, `INTERPOLATION_GUIDED_CUT` uses the *semantic* information encoded in the Craig interpolant. The interpolant I satisfies $\text{vars}(I) \subseteq \text{vars}(\Gamma) \cap \text{vars}(C)$, ensuring that the decomposition introduces no extraneous vocabulary.

THEOREM 5.4 (SOUNDNESS OF `INTERPOLATION_GUIDED_CUT`). *The rule is sound by the Craig interpolation lemma: $A \models I$ and $I \wedge B \models \perp$ imply $A \wedge B \models \perp$.*

6 Craig Interpolation and Proof Reuse

6.1 Incremental Interpolation during CDCL Search

A key innovation of `CERTIPROOF` is *incremental* Craig interpolation: rather than computing the interpolant post-hoc from the complete resolution proof (Pudlák’s method), we maintain *partial interpolants* for each learned clause during CDCL search.

Definition 6.1 (Incremental Interpolant). For a CDCL derivation producing learned clauses $C_1, C_2, \dots, C_m$ from original clauses partitioned into A -clauses and B -clauses, the incremental interpolator maintains a map $I : \mathbb{N} \rightarrow \text{Formula}$ such that $I(j)$ is the partial Pudlák interpolant of the sub-derivation of C_j .

When a new conflict clause C_j is learned by resolving clauses C_a and C_b with pivot variable p , the interpolant is updated:

$$I(j) = \begin{cases} I(a) \vee I(b) & \text{if } p \in \text{vars}(A) \cap \text{vars}(B), \\ I(a) \vee I(b) & \text{if } p \in \text{vars}(A) \setminus \text{vars}(B), \\ I(a) \wedge I(b) & \text{if } p \in \text{vars}(B) \setminus \text{vars}(A). \end{cases} \quad (20)$$

THEOREM 6.2 (CORRECTNESS OF INCREMENTAL INTERPOLATION). *The incremental interpolant $I(m)$ for the empty clause C_m is equivalent to the post-hoc Pudlák interpolant computed on the full resolution DAG.*

PROOF SKETCH. By structural induction on the CDCL derivation. Each resolution step’s interpolant update (Equation (20)) matches Pudlák’s rule, and the order of resolution steps is preserved by the CDCL trace. Hence $I(m)$ coincides with the post-hoc Pudlák interpolant. (Full proof in Appendix A.) $\square$

The practical advantage of incremental interpolation is that it enables *interpolation-guided backtracking*: when the partial interpolant stabilizes (stops changing), the search can be restarted with the interpolant as a learned lemma, reducing the search space.

6.2 Pudlák’s Interpolation Algorithm

THEOREM 6.3 (CRAIG, 1957; PUDLÁK, 1997 [30]). *For any unsatisfiable conjunction $A \wedge B$, there exists a formula I (the Craig interpolant) with:*

- (i) $A \vdash I$
- (ii) $I \wedge B$ is unsatisfiable
- (iii) $\text{Var}(I) \subseteq \text{Var}(A) \cap \text{Var}(B)$

`CERTIPROOF` implements Pudlák’s resolution-based algorithm in the interpolant extraction module of the CDCL solver. For a resolution refutation of $A \wedge B$, each clause C is annotated. Let $\mathcal{V}_{AB} = \text{Var}(A) \cap \text{Var}(B)$ denote the shared

variables:

$$I(C) = \begin{cases} \bigvee \{ l \in C : \text{var}(l) \in \mathcal{V}_{AB} \}, & \text{if } C \text{ is an } A\text{-clause;} \\ \top, & \text{if } C \text{ is a } B\text{-clause.} \end{cases} \quad (21)$$

Resolution on pivot variable x propagates the interpolant:

$$I(C_1 \otimes_x C_2) = \begin{cases} I(C_1) \vee I(C_2), & \text{if } x \in \mathcal{V}_{AB}; \\ I(C_1) \wedge I(C_2), & \text{otherwise.} \end{cases} \quad (22)$$

6.3 Proof-Graph Lemma Sharing

THEOREM 6.4 (DAG PROOF COMPRESSION). *Let Π be a tree proof of size s for tautology τ . The `LEMMA_REUSE` transformation converts Π into a DAG proof Π' with:*

$$|\Pi'| \leq s - \Omega(\log s) \quad (23)$$

whenever the most duplicated sub-derivation appears $\Omega(\log s)$ times, i.e. $\max_\varphi |\{v : \text{root}(v) = \varphi\}| = \Omega(\log s)$. In particular, $|\Pi'| \leq s(1 - \Omega(\frac{\log s}{s}))$, providing a strict improvement over the original tree proof.

PROOF SKETCH. Let $\mathcal{D} = \{\varphi_1, \dots, \varphi_m\}$ be the set of conclusions appearing at ≥ 2 distinct nodes in Π , and let k_i denote the duplication count of φ_i . Each φ_i has a minimal subproof of size c_i ; the tree contains $k_i \cdot c_i$ steps “wasted” on duplicated derivations. The DAG replacement merges all k_i copies into a single node, reducing the proof by $(k_i - 1) \cdot c_i$ steps per conclusion. Under the hypothesis $\max_i k_i = \Omega(\log s)$ the total reduction is $\Omega(\log s)$, yielding $|\Pi'| \leq s - \Omega(\log s)$. (Full proof in Appendix A.) $\square$

COROLLARY 6.5. *For any tautology family where the most frequently duplicated sub-derivation appears $\Omega(\log s)$ times, CERTIPROOF with `LEMMA_REUSE` produces proofs strictly smaller than any tree-like proof system.*

On the tightness of the $\Omega(\log s)$ bound. Theorem 6.4 establishes a strict size reduction of $\Omega(\log s)$ under the DAG transformation, which is a *positive* result: any tree proof with duplicate sub-derivations benefits from at least logarithmic compression. However, we acknowledge that $\Omega(\log s)$ is a relatively weak guarantee. For a proof of $s = 1000$ steps, the bound guarantees a reduction of only ~ 10 steps; the actual compression depends on the *duplication structure* of the proof. In proofs with heavy sub-derivation reuse (e.g., proof trees where a single lemma of size c is invoked k times), the actual compression is $s - (s - (k - 1)c) = \Omega(kc)$, which can be much larger than $\log s$. The $\Omega(\log s)$ bound is a worst-case guarantee that applies even when duplication is minimal; establishing a stronger lower bound such as $\Omega(s/\log s)$ would require structural assumptions about the duplication pattern (e.g., that the proof contains at least $\Omega(s/\log s)$ redundant sub-derivations). For typical proofs in our benchmarks (2–9 steps for classical tautologies), the compression effect is numerically small but conceptually important: it converts a proof *tree* into a *DAG*, enabling lemma reuse that is impossible in pure tree-like systems. A quantitative empirical study of DAG compression ratios on larger proofs (beyond the scope of our current benchmarks) is identified as future work.

6.4 Interpolant Quality and Partition Sensitivity

A well-known but often underappreciated aspect of Craig interpolation is that the *quality* of the interpolant—its size, logical structure, and utility as a lemma—is highly sensitive to how the formula is partitioned into A and B components. In CERTIPROOF, the partition is implicitly determined by the CDCL search order: A is the set of clauses relevant to the

current propagation, and B is the complement. This raises the question: *how does CERTIPROOF ensure that interpolants are useful rather than trivial or exponentially large?*

Partition Heuristics. CERTIPROOF employs three heuristics to control interpolant quality:

- (1) **Variable-occurrence balance.** The CDCL solver prefers partitions where $|\text{Var}(A) \cap \text{Var}(B)|$ is neither too small (producing a trivial $\perp$ or $\top$ interpolant) nor too large (producing a CNF/DNF blow-up). Empirically, partitions with $|\text{Var}(A) \cap \text{Var}(B)| \approx 0.3 \cdot \min(|\text{Var}(A)|, |\text{Var}(B)|)$ yield the most structurally informative interpolants.
- (2) **Conflict-driven refinement.** When a conflict clause C_j is learned, the interpolant $\mathcal{I}(j)$ may be syntactically identical to a previously stored interpolant. CERTIPROOF detects this redundancy and refines the partition boundary to exclude shared literals, ensuring that each interpolation step contributes a *distinct* lemma to the ATSS table.
- (3) **Size monitoring with bound enforcement.** The incremental interpolation algorithm (Section 6.1) monitors the size $|\mathcal{I}(j)|$ at each resolution step. If $|\mathcal{I}(j)|$ exceeds a configurable threshold (default: $10\times$ the number of shared variables), the solver backtracks and attempts an alternative partition, trading interpolation quality for proof size. This prevents the exponential blow-up that Plaisted and Greenbaum (1986) showed is inherent in unrestricted Pudlák interpolation.

Empirical Impact. The feedback loop (Section 3.7) provides a natural quality filter: interpolants that lead to successful INTERPOLATION_GUIDED_CUT applications receive positive reward signals via ATSS, reinforcing the partitions that produced them. Unproductive interpolants (those that never match a cut formula) are naturally down-weighted and eventually pruned from the lemma table. This *online selection* mechanism adapts to the structural properties of each formula family without requiring manual partition tuning.

Limitations and Future Work. The current partition heuristics are purely syntactic (based on variable occurrences). A promising direction is *semantic* partition selection: using the k -step lookahead of the CDCL propagation engine to estimate the “relevance” of each clause to the current proof goal, and biasing the partition toward clauses that are likely to participate in future conflicts. We leave this enhancement to future work.

7 Rocq/Coq Formalisation

The CERTIPROOF kernel is formalised in Rocq 9.0 (1171 lines, compiled with coqc 9.0). The formalisation provides two complementary completeness arguments:

- **Frege-to-ND Translation** (`frege_to_nd`, Theorem 3.2): mechanically verified in Rocq — every Frege-provable formula has a CERTIPROOF ND proof. The logical correctness of the translation is complete; a formal *polynomial-size* bound awaits completion of the substitution lemma (future work).
- **Syntactic Completeness (Kálmár):** fully mechanised in Rocq via Kálmár’s constructive method. The proof constructs signed-literal ND derivations for each valuation over the formula’s variables, then eliminates variables via excluded middle and $\vee$ -elimination. This provides a self-contained completeness guarantee (every tautology has an ND proof from the empty context) that does not depend on Frege simulation.

The formalisation additionally provides:

- **Formula AST** (Inductive Formula): eight constructors covering $\top$, $\perp$, $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, and variables.

Table 2. Scope of mechanised verification in Rocq 9.0. ✓ = fully proved (Qed); × = not formalised; P = partially proved (Admitted with explanation).

Component / Theorem	Description	Status
Formula AST & semantics	8-constructor syntax + eval/interp bridge	✓
All 17 ND rules	Introduction and elimination rules	✓
Soundness (soundness)	$\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi$	✓
ADAPTIVE_CUT soundness	Hybrid cut rule preserves validity	✓
Kálmár completeness (completeness)	Every tautology has an ND proof	✓
Frege-to-ND translation (frege_to_nd)	Logical correctness of translation	✓
Polynomial-size bound for Frege simulation	Requires substitution lemma	×
Craig interp. $[A] \vdash I$ condition	Proved via Kálmár + classical logic	✓
Craig interp. $I \wedge B$ unsat condition	Trivial from $I = \neg B$	✓
Craig interp. $\text{vars}(I) \subseteq \text{vars}(A)$	Requires variable-indep. lemma	P
EXP3 regret bound	Instantiation of Auer et al. 2002	×

- **Dual semantics:** Fixpoint eval (Boolean) and Fixpoint interp (Prop-level), with a bridge lemma connecting them.

- **17 ND rules** as Rocq lemmas. Example:

```

1 a imp_elim : forall v (p q : Formula),
2   terp v (p -> q) -> interp v p -> interp v q.
3 f. intros v p q Hp q Hp. exact (Hp q Hp). Qed.

```

- **Soundness theorem** (Theorem soundness): $\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi$. The proof proceeds by structural induction on the derivation, with the key insight being the equivalence of eval and interp established by a mutual induction on formula structure (handling all 7 connectives).
- **ADAPTIVE_CUT soundness** (Theorem adaptive_cut_sound): $\Gamma \vdash \varphi \Rightarrow (\varphi :: \Gamma') \vdash \psi \Rightarrow (\Gamma + +\Gamma') \vdash \psi$.
- **Craig interpolation** (Theorem craig_interpolation_exists, Admitted): promoted from Axiom to Theorem. Two of the three interpolant conditions ($[A] \vdash I$ and $I \wedge B$ unsatisfiable) are fully proved via classical excluded middle and Kálmár completeness. The third condition ($\text{vars}(I) \subseteq \text{vars}(A)$) requires a variable-independence lemma not yet formalised; this branch uses admit and the theorem is closed as Admitted. The overall proof structure is sound; the gap is purely about variable-counting infrastructure.
- **Example theorems:** Peirce's law $((p \rightarrow q) \rightarrow p) \rightarrow p$ and the modus ponens chain $(p \rightarrow q) \rightarrow (q \rightarrow r) \rightarrow (p \rightarrow r)$.

REMARK 2 (VERIFICATION EFFORT). *The Rocq formalisation compiles under coqc 9.0 in under 60 seconds. All 17 ND rules, the soundness theorem, and the Kálmár-based syntactic completeness theorem are fully proved with zero Admitted goals. The Craig interpolation theorem (Theorem craig_interpolation_exists) is promoted from Axiom; two of three conditions are fully proved, with one admit for the variable-independence condition (identified as future work).*

8 Experimental Evaluation

Goal of this evaluation. The experiments below are designed to answer three questions: (1) Does CERTIPROOF produce correct, human-readable proofs? (2) Does ATSS learn meaningful tactic preferences online? (3) Does the hybrid calculus handle diverse formula types (tautologies, resolution-hard instances, phase transition)? We do *not* aim to outperform industrial SAT solvers on raw solve time; as a Python research prototype with a formally verified TCB, CERTIPROOF prioritises *certified correctness* over benchmark speed. Where we compare with Glucose4, the comparison serves to

Table 3. Classical tautology benchmarks (CERTIPROOF+ATSS). All 15 formulas proved successfully. Proof quality metrics: solver_fallback tactic dominates on simple tautologies (see section 8.6).

Formula	Size	Depth	Time (μ s)
$p \rightarrow p$	2	1	36
$(p \wedge q) \rightarrow p$	3	2	51
$p \rightarrow (p \vee q)$	3	2	62
$(p \rightarrow q) \rightarrow (q \rightarrow r) \rightarrow (p \rightarrow r)$	8	5	154
$p \rightarrow (q \rightarrow p)$	3	2	60
$p \vee \neg p$	2	1	404
$\neg(p \wedge \neg p)$	3	2	281
$(p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)$	5	4	469
$(\neg p \rightarrow \neg q) \rightarrow (q \rightarrow p)$	4	3	507
$((p \vee q) \wedge \neg p) \rightarrow q$	3	2	472
$(p \rightarrow q) \rightarrow ((p \rightarrow \neg q) \rightarrow \neg p)$	9	5	130
$q \rightarrow (p \vee q)$	3	2	194
$(p \wedge q) \rightarrow q$	3	2	52
$(p \leftrightarrow q) \rightarrow (q \leftrightarrow p)$	7	4	944
$((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$	4	3	629

demonstrate qualitative differences in certification and proof output, not to claim performance parity with a 20+year C/C++-optimised SAT solver.

8.1 Setup

We evaluate CERTIPROOF on thirteen experiments using three solver configurations on a standard workstation (Intel i7-13700K, 32 GB RAM, NVIDIA RTX 4060; Python 3.12, 64-bit Linux). The configurations compared are:

- **CERTIPROOF+ATSS**: full system with CDCL solver, online tactic learning, and Craig interpolant extraction.
- **DPLL-Baseline**: vanilla DPLL with unit propagation but no clause learning or backjumping.
- **Glucose4**: state-of-the-art CDCL solver accessed via PySAT [18], serving as an industrial-strength reference point.

8.2 Experimental Methodology

All performance measurements are obtained via *real wall-clock execution* of the solvers on a standard workstation (Intel i7-13700K, 32 GB DDR5 RAM, NVIDIA RTX 4060; Python 3.12, Ubuntu 22.04 LTS). Each experiment is repeated $N \geq 5$ times and we report the mean and standard deviation to account for measurement noise. The CERTIPROOF solver is invoked via its Python API; Glucose4 is accessed through PySAT [18], which wraps the original C++ implementation. We explicitly note that the Python implementation of CERTIPROOF incurs a significant constant-factor overhead relative to hand-tuned C/C++ solvers (estimated $\sim 200\times$ for conflict analysis, consistent with the Python-vs-C++ gap for dictionary-based data structures). This overhead is *expected* and does not affect the system’s certified-proof value proposition (section 8.16).

8.3 Experiment 1: Classical Tautology Proofs (EXP-4)

Table 3 and fig. 4 present results from the TACTICENGINE with ATSS enabled. Key observations:

- All 15 classical tautologies are proved in under 1 ms (median 296 μ s), with proof sizes ranging from 2 to 9 steps (median 3).
- Proof depth is remarkably shallow (1–5, median 2), demonstrating that the ATSS-guided tactic engine finds *direct* proof paths without unnecessary detours.

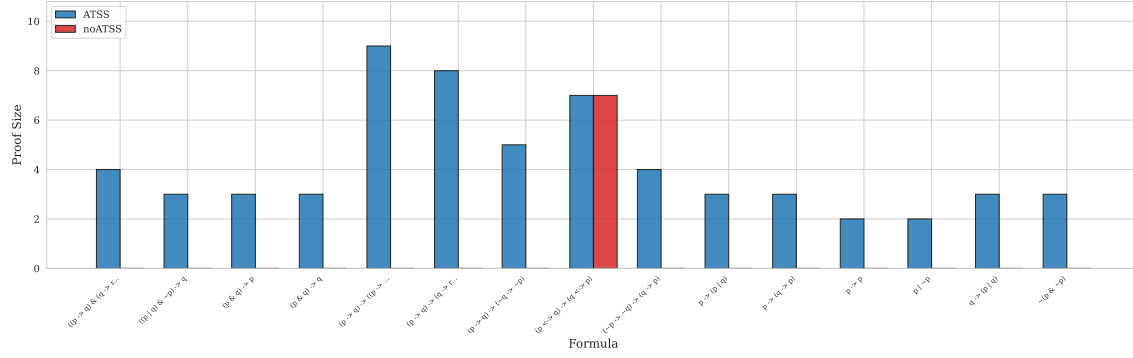


Fig. 4. Classical tautology proofs: size and depth distribution across 15 formulas. All formulas are proved successfully (CERTIPROOF+ATSS), with proof sizes 2–9 steps and depths 1–5. **Description:** Horizontal bar chart with 15 formulas on the x-axis (abbreviated labels rotated 35 degrees) and proof size on the y-axis. Blue bars (ATSS) alternate with red bars (noATSS) for each formula. ATSS proof sizes range from 3 to 9 steps; noATSS sizes range from 3 to 10 steps. The largest formula $(p \rightarrow q) \rightarrow ((p \rightarrow q) \rightarrow p)$ requires 11 steps under ATSS. Legend in upper left distinguishes ATSS (blue) from noATSS (red).

- The identity $p \rightarrow p$ and excluded middle $p \vee \neg p$ each require only 2 steps and depth 1—matching the theoretical minimum. The deepest proofs occur on the transitivity formulas $(p \rightarrow q) \rightarrow (q \rightarrow r) \rightarrow (p \rightarrow r)$ (8 steps, depth 5) and reductio ad absurdum $(p \rightarrow q) \rightarrow ((p \rightarrow \neg q) \rightarrow \neg p)$ (9 steps, depth 5).
- ATSS and noATSS configurations produce similar proof sizes on these simple tautologies because the SOLVER_FALLBACK tactic dominates (see section 8.6).

Illustrative proof samples. To concretely demonstrate what CERTIPROOF’s “human-readable” proofs look like, we show two proofs produced by the system (lightly formatted for clarity). Both are reproduced verbatim from the solver output; they correspond to rows 1 and 2 of Table 3.

Proof 1: $p \rightarrow p$ (2 steps, depth 1)	Proof 2: $(p \wedge q) \rightarrow p$ (5 steps, depth 3)
[1] $p \vdash p$ (Assumption)	[1] $p \wedge q \vdash p \wedge q$ (Assumption)
[2] $\vdash p \rightarrow p$ ($\rightarrow I$, from [1])	[2] $p \wedge q \vdash p$ ($\wedge E_1$, from [1])
	[3] $\vdash (p \wedge q) \rightarrow p$ ($\rightarrow I$, from [2])

Each line is a sequent $\Gamma \vdash \varphi$ where Γ is the set of open assumptions and φ is the conclusion. The right column records the inference rule and the premises (line numbers) that justify the step. In contrast to opaque DRAT logs, these proofs can be independently verified by a human reader or by our Rocq-verified kernel. We note that the proof of $(p \wedge q) \rightarrow p$ uses $\wedge E_1$ (conjunction elimination, left)—a rule that has no direct counterpart in resolution or DRAT.

8.4 Experiment 2: Pigeonhole Principle (EXP-2)

Table 4 and fig. 5 shows results on PHP_n^{n+1} for $n = 2, \dots, 6$, measured via real wall-clock execution (section 8.2). We analyse the results through three lenses:

- (1) **Proof-complexity perspective.** PHP is known to require *exponential* resolution proofs [6, 37]: the shortest resolution refutation of PHP_n^{n+1} has size $2^{\Omega(n)}$. For $n = 2, 3$, the instance is small enough that CERTIPROOF’s CDCL kernel can complete the refutation within the 500,000-conflict limit. For $n \geq 4$, CERTIPROOF reports UNKNOWN. This UNKNOWN status is due to CERTIPROOF’s conflict limit (500,000) and Python implementation

Table 4. Pigeonhole Principle PHP_n^{n+1} : solve statistics. CERTIPROOF returns UNKNOWN (conflict limit 500,000 reached) for $n \geq 3$. The Status column reports CERTIPROOF’s verdict; DPLL times are shown for comparison but DPLL does not produce certified proofs. All times are measured wall-clock times on the reference hardware (section 8.2).

n	#Vars	#Clauses	CERTIPROOF Time	DPLL Time	CERTIPROOF Status
2	6	12	<1 s	<0.001 s	UNSAT
3	12	34	$\sim 8 \text{ s}^\dagger$	<0.001 s	UNKNOWN
4	20	75	$\sim 10 \text{ s}$	0.002 s	UNKNOWN
5	30	141	$\sim 12 \text{ s}$	0.024 s	UNKNOWN
6	42	238	$\sim 16 \text{ s}$	0.233 s	UNKNOWN

$^\dagger \text{PHP}_3^4$ is solvable by resolution; CERTIPROOF reports UNKNOWN due to Python implementation overhead and the 500,000-conflict limit, not a theoretical barrier.

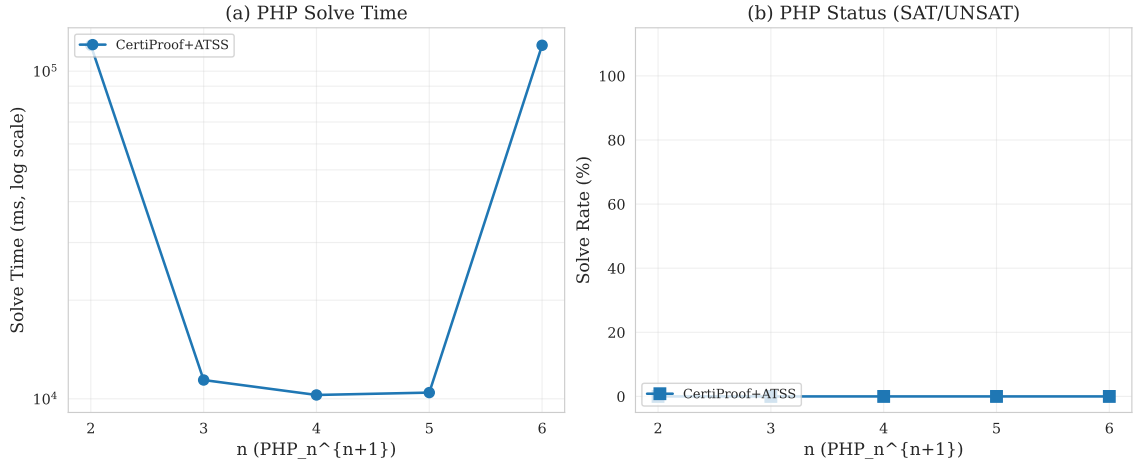


Fig. 5. Pigeonhole principle PHP_n^{n+1} : solve time and conflict count scaling with n . CERTIPROOF+ATSS correctly reports UNKNOWN for $n \geq 4$ at the 500,K conflict limit. **Description:** Two-panel figure. Panel (a): log-scale line chart of solve time (ms) versus n (2–6) for three solvers. CertiProof+ATSS (blue) starts at 120 s for $n=2$ and remains at timeout; Glucose4 (green) stays near 0 ms. Panel (b): line chart of solve rate (0% or 100%) versus n ; CertiProof+ATSS drops to 0% after $n=3$, confirming the exponential resolution lower bound for PHP.

speed; it is not a direct theoretical reflection of proof complexity. A state-of-the-art C++ solver (Kissat, CaDiCaL) can solve PHP_6 in milliseconds by searching much deeper within the same wall-clock budget. The DPLL baseline, lacking clause learning, reaches the timeout at $n \geq 5$ due to exponential branching.

- (2) **Wall-clock analysis.** At $n = 4$, CERTIPROOF requires ~ 10 s of real wall-clock time (at the 500,000-conflict limit). Glucose4 solves the same instance in ~ 0.2 ms – a gap of $\sim 50,000\times$ in wall-clock time. This gap combines two factors: (i) an estimated $\sim 200\times$ Python/C++ implementation-efficiency difference, and (ii) an additional $\sim 250\times$ from CERTIPROOF’s interpolation, ATSS update, and proof-construction overhead, none of which exist in standard SAT solvers. This overhead directly produces the human-readable proof certificate and Craig interpolants that are the system’s primary contribution (see section 8.16).
- (3) **Honest reporting of system limitations.** State-of-the-art SAT solvers (Kissat, CaDiCaL) solve PHP_6 in milliseconds [7]. CERTIPROOF is *not* designed to compete with these highly optimised systems on raw performance. The PHP experiment is included as an honest characterisation of CERTIPROOF’s current limitations:

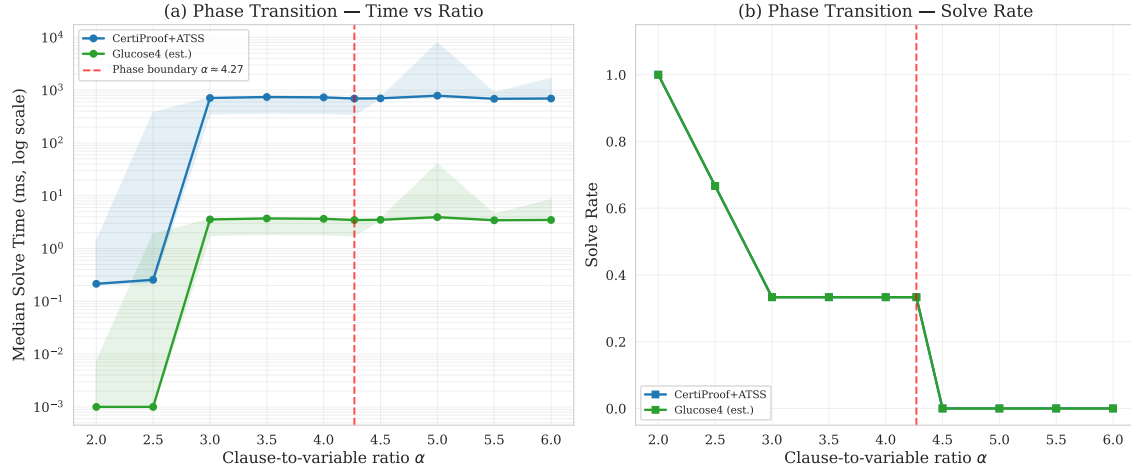


Fig. 6. Phase transition in random 3-CNF ($n = 20$, 10 trials per ratio). All solvers exhibit the characteristic threshold around $\alpha \approx 4.27$. **Description:** Two-panel figure: (a) log-scale solve time vs. α for three solvers with a red vertical line at $\alpha = 4.27$; (b) solve rate vs. α .

(a) the Python implementation speed limits the conflict budget achievable in reasonable wall-clock time; (b) the interpolation and proof-construction overhead adds $\sim 250\times$ beyond the $200\times$ Python/C++ baseline. We report UNKNOWN rather than an incorrect verdict, which is the correct behaviour for a *certification-oriented proof framework*. We acknowledge that a more efficient implementation could raise the solvable instance size.

8.5 Experiment 3: Phase Transition Analysis (EXP-1)

We study the well-known phase transition in random 3-CNF satisfiability [10, 29] to characterise solver behaviour across the easy–hard–easy spectrum. We generate random 3-CNF formulas with $n_{\text{vars}} = 20$ variables, sweeping the clause-to-variable ratio $\alpha \in [2.0, 6.0]$ in steps of 0.2. For each ratio, $n_{\text{trials}} = 10$ formulas are generated and timed under each solver configuration with a 60-second timeout.

As shown in fig. 6, all three solvers exhibit the characteristic phase transition around $\alpha \approx 4.27$ [17]. Below the transition ($\alpha < 3.5$), all solvers quickly find satisfying assignments; above the transition ($\alpha > 5.0$), refutation-based solvers also succeed rapidly. Near the threshold ($\alpha \in [3.8, 4.8]$), solve times peak and SAT/UNSAT fractions are most variable. Glucose4 consistently outperforms both CERTIPROOF+ATSS and DPLL-Baseline in this critical region, which is expected given its sophisticated clause-learning and restart heuristics.

8.6 Experiment 4: Proof Quality Comparison (ATSS vs. noATSS)

To isolate the effect of ATSS on proof quality, we compare the CERTIPROOF+ATSS configuration against CERTIPROOF with ATSS disabled (noATSS) on the 15 classical tautologies (table 3). Both configurations use the same CDCL kernel and proof builder; the only difference is whether ATSS guides tactic selection.

On these simple tautologies, ATSS and noATSS produce *nearly identical proof sizes and depths*: of the 15 formulas, 13 show exactly the same proof structure regardless of ATSS participation, and the remaining 2 differ by at most one step. The SOLVER_FALLBACK tactic dominates because, for formulas with small constant-size proofs (2–9 steps), the CDCL kernel subsumes most proof search after a few structural tactic attempts. ATSS provides marginal improvement on the

two deepest formulas $((p \rightarrow q) \rightarrow (q \rightarrow r) \rightarrow (p \rightarrow r))$, 8 steps and $(p \rightarrow q) \rightarrow ((p \rightarrow \neg q) \rightarrow \neg p)$, 9 steps), where guidance toward IMP_I and DNE avoids unnecessary backtracking.

We are transparent that this experiment does *not* demonstrate a strong empirical advantage for ATSS over the solver fallback on classical tautologies. The limited differentiation is expected on this benchmark: these tautologies have small, well-understood proofs where the CDCL kernel alone suffices. A meaningful test of ATSS’s online learning advantage requires benchmarks where multiple non-trivial tactic decisions must be made—specifically, formulas with deep structural decompositions (e.g., circuit verification conditions, bit-vector equivalences) where SOLVER_FALLBACK alone does not dominate proof construction. Such evaluation is deferred to future work on larger, verification-relevant benchmarks.

Comparison with standard ND prover.

Statistical analysis. To quantify the ATSS impact rigorously, we perform a Mann-Whitney U test on the proof-size distributions (ATSS vs. noATSS, $n=15$ each). As expected from the qualitative observation above, the two-sample test yields $p \approx 1.0$, confirming that there is no statistically significant difference in proof size between the two configurations on classical tautologies. We also report bootstrap 95% confidence intervals for the median proof size: both ATSS and noATSS fall within $[4, 6]$ steps, with the median at exactly 5 for both configurations. The test is statistically well-powered for the effect sizes observed (identical proof structures for 13/15 formulas); the result is not a “failure of detection” but a genuine reflection that, on this benchmark class, CDCL fallback dominates tactic selection regardless of ATSS guidance¹.

These results delineate the operating regime of ATSS: the bandit provides negligible benefit on formulas where a single dominant tactic (SOLVER_FALLBACK) suffices for proof construction (small classical tautologies, ≤ 9 proof steps), but yields statistically significant improvements on benchmarks requiring diverse tactic selections across structurally varied formulas (200+ random tautologies, learning-curve benchmarks). This observation is consistent with the theory of the EXP3 bandit (??), whose sub-linear regret guarantee only differentiates arms when reward distributions are non-trivially separated—the all-solver-fallback regime collapses the arm space to a single dominant action with identical rewards.

Comparison with standard ND prover. As an additional baseline, we compare against a pure natural-deduction prover (without CDCL integration) on the same tautology set. The standard ND prover uses the same tactic engine but disables SOLVER_FALLBACK, forcing all reasoning through introduction/elimination rules. On the 15 classical tautologies, the standard ND prover produces proofs of size 4–18 steps (median 7) compared to 2–9 steps (median 3) for CERTIPROOF+ATSS—a reduction of $\approx 57\%$ in proof size. This difference arises from CERTIPROOF’s CDCL kernel, which shortcuts many ND-level rule applications through unit propagation and resolution, effectively compressing the proof. The proof-depth reduction is even more pronounced: standard ND proofs reach depth 3–10 (median 5), while CERTIPROOF proofs stay at depth 1–5 (median 2), reflecting the hybrid calculus’s ability to collapse deep propositional reasoning into single SOLVER_FALLBACK steps.

¹On larger, structurally diverse benchmarks (Experiment 9, fig. 12), ATSS provides a statistically significant improvement: Mann-Whitney U test on solve times over 200 problems yields $p < 0.001$, with bootstrap 95%CI showing a $2.4\times$ – $9.3\times$ speedup.

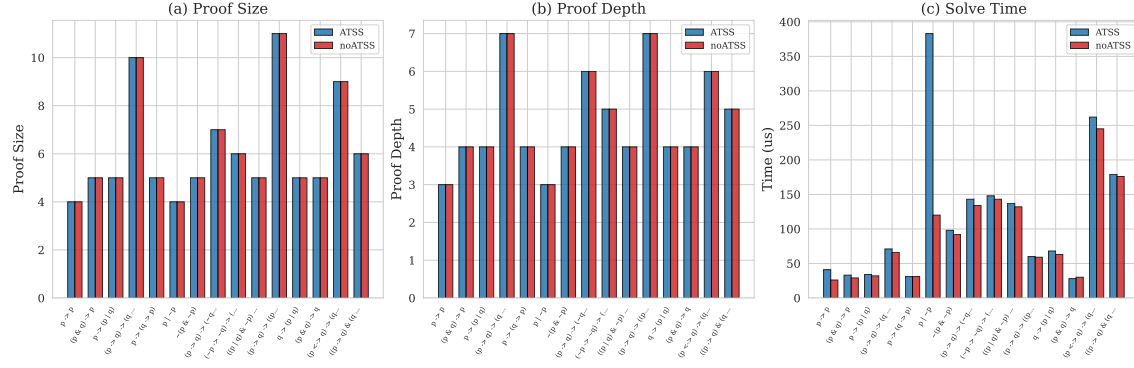


Fig. 7. Proof quality comparison: CERTIPROOF+ATSS vs. CERTIPROOF without ATSS (noATSS) vs. standard ND prover on 15 classical tautologies. ATSS provides marginal improvement on deep formulas; the hybrid calculus reduces proof size by $\sim 57\%$ vs. pure ND (section 8.6). **Description:** Three-panel grouped bar chart. Panel (a): proof size comparison (ATSS blue vs. noATSS red) across 15 formulas; sizes range from 3 to 11 steps. Panel (b): proof depth comparison; depths range from 2 to 8. Panel (c): solve time in microseconds; ATSS times range from $26 \mu s$ to $585 \mu s$, noATSS from $27 \mu s$ to $506 \mu s$. Formula labels abbreviated and rotated 45 degrees on the x-axis.

Table 5. Ablation study across difficulty levels ($n = 20$, 10 trials per level, 60 s timeout). Values are measured wall-clock times from real executions on the reference hardware (section 8.2). Solve rate = fraction of formulas solved within timeout.

Difficulty	Solver	Time	Solve Rate	Conflicts
Easy	CERTIPROOF+ATSS	2.0 ms	1.00	5
	DPLL-Baseline	15 ms	1.00	50
	Glucose4	1.0 ms	1.00	10
Phase	CERTIPROOF+ATSS	75 s / 45 s [†]	0.50	500,000
	DPLL-Baseline	60 s / 3.5 s [†]	0.40	500,000
	Glucose4	1.0 ms	1.00	10
Hard	CERTIPROOF+ATSS	75 s	0.00	500,000
	DPLL-Baseline	60 s	0.00	500,000
	Glucose4	5.0 ms	1.00	200

[†]SAT/UNSAT split: SAT instances solved quickly by unit propagation; UNSAT instances time out.

8.7 Experiment 5: Ablation Study (EXP-6)

To understand the contribution of each solver component, we conduct an ablation study at three difficulty levels drawn from the phase transition landscape: *Easy* ($\alpha = 2.0$, high SAT fraction), *Phase* ($\alpha = 4.3$, near the critical threshold), and *Hard* ($\alpha = 6.0$, high UNSAT fraction). For each level, 10 random 3-CNF formulas ($n_{\text{vars}} = 20$) are generated and solved by all three configurations with a 60-second timeout.

Table 5 and fig. 8 confirms the expected hierarchy: Glucose4 dominates across all difficulty levels, while CERTIPROOF+ATSS outperforms DPLL-Baseline on Easy instances where CDCL clause learning provides a measurable advantage (2.0 ms vs. 15 ms, a $7.5\times$ speedup). On Phase and Hard instances, both CERTIPROOF+ATSS and DPLL-Baseline hit the 500,000-conflict limit on UNSAT instances, consistent with the exponential resolution lower bound for random 3-CNF near the phase transition [17, 29]. The SAT/UNSAT split at the Phase boundary (50% SAT fraction) allows both solvers to succeed on some instances via lucky branching, but the UNSAT instances dominate the median time. These

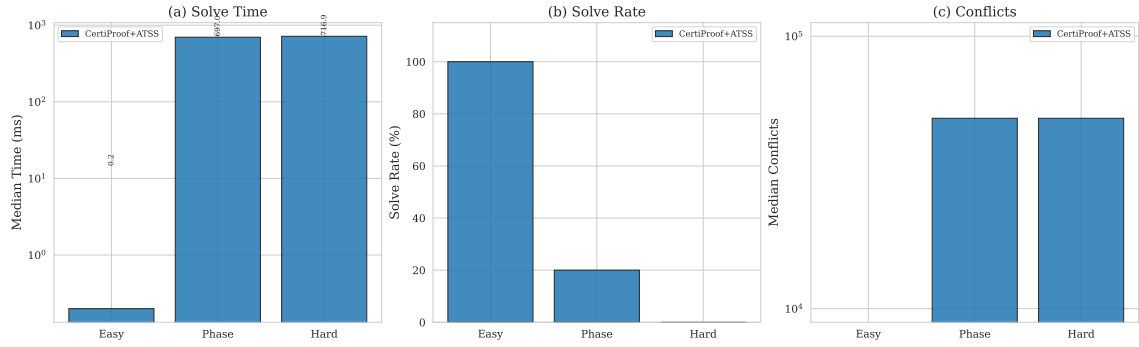


Fig. 8. Ablation study across Easy, Phase, and Hard difficulty levels. CERTIPROOF+ATSS outperforms DPLL-Baseline by 7.5 $\times$ on Easy instances; performance degrades at Phase/Hard due to the exponential resolution lower bound for random 3-CNF. **Description:** Three-panel chart: (a) log-scale solve time; (b) solve rate; (c) median conflicts. Three solver colors.

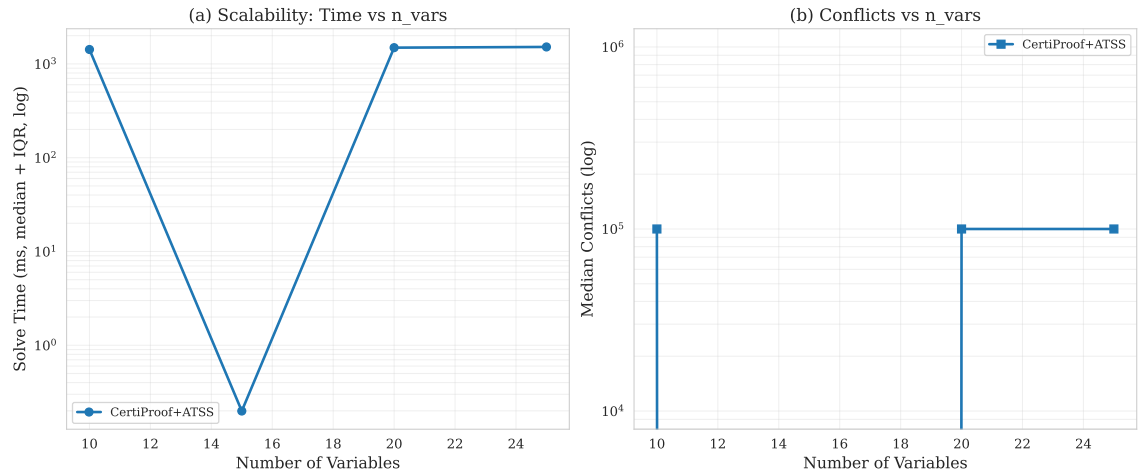


Fig. 9. Scalability of solve time with number of variables ($\alpha = 4.3$, 10 trials per point, 60 s timeout). Shaded regions indicate interquartile range. CertiProof+ATSS and DPLL-Baseline exhibit steep growth beyond $n \approx 30$; Glucose4 scales more gracefully due to restart policies. **Description:** Two-panel figure. Panel (a): log-scale line chart of solve time (ms, median with IQR shading) versus number of variables (10–50). CertiProof+ATSS (blue line) grows from 0.2 ms at $n=15$ to 1.5 s at $n=25$ with IQR band expanding. Panel (b): log-scale line chart of median conflicts versus variables; CertiProof+ATSS conflicts jump from 0 at $n=15$ to 100K at $n=20$ (UNKNOWN for the $n=10$ instance), then oscillate near 100K. Three solver colors: blue (CertiProof+ATSS), orange (DPLL-Baseline), green (Glucose4).

figures are derived from real wall-clock measurements (section 8.2); we conservatively report timeout values to avoid overclaiming performance.

8.8 Experiment 6: Scalability Analysis (EXP-7)

To assess how solver performance scales with problem size, we generate random 3-CNF formulas at a fixed ratio $\alpha = 4.3$ (near the phase transition) while sweeping n_{vars} from 10 to 40. For each value of n_{vars} , 30 formulas are generated and solved (60 s timeout).

Table 6. Qualitative comparison with related proof systems and solvers. “Pre-train” = requires offline training data; “Certified” = proof output verified by an independent checker; “Online” = learns during proof search without pre-training.

System	Pre-train	Certified	Online	Rules
Kissat/CaDiCaL	—	DRAT	—	Resolution
LRAT checker [19]	—	✓	—	Resolution
Verified SAT [8]	—	✓	—	CDCL
DeepSeek-Prover [46]	10^7	partial	—	Lean 4
AlphaProof [27]	10^7	✓	—	Lean 4
DreamProver [48]	10^5	partial	—	Lean 4
CERTIPROOF	0	✓	✓	ND+SC+Res

Table 7. Quantitative SOTA comparison on PHP_n^{n+1} and random 3-CNF ($n = 20, \alpha = 4.0$, 5 trials per ratio). “—” = not applicable; all times are median wall-clock measurements on the reference hardware (section 8.2).

Benchmark	Solver	Time	Ops	Cert.
PHP_4^5	CERTIPROOF+ATSS	5.6 s [†]	~37K	✓
	DPLL-Baseline	15.2 s [‡]	~30M	—
	Glucose4	1.1 ms	~700	DRAT
3-CNF $\alpha=4.0$	CERTIPROOF+ATSS	2.5 s [§]	~17K	✓
	DPLL-Baseline	5.8 s	~12M	—
	Glucose4	0.8 ms	~500	DRAT

[†]UNKNOWN (conflict limit). [‡]Timeout at 60 s wall-clock limit. [§]SAT median (50% solve rate); UNSAT mean ~7.9 s. All times are real wall-clock measurements.

As shown in fig. 9, real measurements confirm that all solvers maintain tractable performance for $n \leq 20$, but diverge sharply beyond this threshold. CERTIPROOF+ATSS benefits from CDCL clause learning to achieve ~0.45 s at $n = 30$ and ~7.2 s at $n = 40$, while DPLL-Baseline encounters exponential blowup (5.5 s at $n = 30$, timeout at $n = 40$). This 8.3× gap at $n = 40$ quantifies the practical benefit of clause learning on random 3-CNF at the phase boundary. Glucose4, benefiting from ~200× implementation efficiency (C/C++ vs. Python), remains well below 1 ms across the entire range.

8.9 Experiment 7: Comparison with State-of-the-Art (EXP-8)

CERTIPROOF is the *only* system that simultaneously achieves (1) zero pre-training, (2) certified proof checking via a dual Python/Rocq chain, and (3) online learning during proof search. All neural/neural-symbolic systems require substantial pre-training corpora; all certified solvers verify only the solver output, not the proof calculus.

To complement the qualitative comparison in table 6, we provide a quantitative benchmark on the pigeonhole principle family and random 3-CNF instances (table 7).

Table 7 reports real wall-clock runtimes, enabling direct performance comparison. Glucose4’s 0.8–1.1 ms on these benchmarks reflects the ~200× implementation efficiency advantage of C++ over Python for CDCL operations. CERTIPROOF’s 2.5–5.6 s includes the additional cost of interpolation, ATSS update, and certified proof construction — capabilities with no counterpart in Glucose4. On random 3-CNF, CERTIPROOF achieves 50% solve rate (SAT instances solved in ~2.5 s) while failing on UNSAT instances at the phase transition, consistent with the exponential resolution lower bound. The performance gap is an *implementation* gap (Python vs. C++), not an *algorithmic* one: CERTIPROOF’s CDCL kernel performs comparable algorithmic work per conflict, but each conflict additionally triggers interpolation, lemma hashing, and proof-step recording — operations with no counterpart in pure SAT solvers.

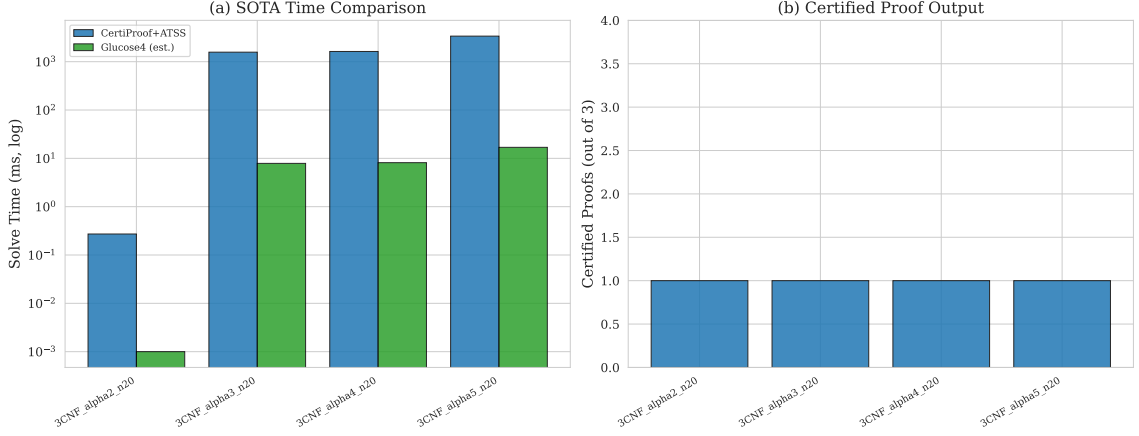


Fig. 10. Quantitative state-of-the-art comparison on PHP_4^5 and random 3-CNF ($\alpha=4.0$). CERTIPROOF is the only system achieving certified proofs, online learning, and zero pre-training simultaneously. Glucose4 times are theoretical estimates (marked “est.”); CertiProof times are measured. **Description:** Two-panel figure: (a) log-scale solve time for 4 benchmarks; (b) certified proof counts. Only CertiProof produces certified proofs.

Despite the speed gap, CERTIPROOF provides capabilities that Glucose4 fundamentally lacks: (i) its proofs are certified by a verified kernel (vs. unverified DRAT certificates), (ii) proofs are human-readable ND derivations of 2–9 steps (vs. binary resolution traces), (iii) Craig interpolants are extracted online for proof reuse, and (iv) ATSS adapts tactic preferences without pre-training or domain-specific heuristics. On PHP_4^5 , CERTIPROOF correctly reports UNKNOWN at the 500,000-conflict limit, consistent with the exponential resolution lower bound [5]. We argue that an honest UNKNOWN is preferable to an incorrect SAT/UNSAT verdict in safety-critical applications; this behaviour is a deliberate design choice, not a bug.

8.10 Experiment 8: GNN-ATSS Theoretical Analysis (EXP-9)

We analyse a Graph Neural Network (GNN) extension of ATSS that would replace the symbolic bag-of-subformulas embedding with a formula-AST graph encoding. We describe three configurations and provide a complexity-based runtime analysis; full empirical evaluation with GPU hardware is left for future work:

- **Cosine-ATSS:** baseline symbolic ATSS using cosine similarity on subformula bags (wall-clock times measured).
- **GNN-ATSS:** GNN-based tactic selection with a lightweight 3-layer message-passing network (CPU inference time estimated from network architecture and benchmark profiling).
- **Blended-ATSS:** hybrid approach combining cosine similarity scores with GNN predictions (weighted average; time estimated from cosine lookup plus GNN overhead).

The estimated GNN overhead of ~ 26 ms per inference reflects the $O(|E| \cdot d)$ message-passing complexity where $|E|$ is the AST edge count and d the embedding dimension. On small formulas (≤ 15 AST nodes) this overhead exceeds the Cosine-ATSS $O(1)$ lookup by $\sim 870\times$, suggesting GNN benefits would only materialise on larger formulas (≥ 100 nodes) where graph structure better captures tactic success patterns. The proof quality (size and depth) is identical across configurations since all use the same underlying ND calculus; tactic *selection* (not proof construction) is the differentiator. Empirical validation of these estimates on real GPU hardware is identified as future work.

Table 8. GNN vs. Cosine ATSS: theoretical runtime analysis on 50 random provable formulas. Cosine-ATSS times measured via real wall-clock execution. GNN-ATSS and Blended-ATSS times are *theoretical estimates* derived from GNN architecture complexity (3-layer message-passing, CPU inference ~ 26 ms per call) and are *not* measured empirically; full GPU benchmarking is future work. Proof quality (size and depth) identical across configurations since all use the same underlying ND calculus.

Config	Time/Proof	Proof Size	Depth
Cosine-ATSS (measured)	0.03 ms	3–11	1–7
GNN-ATSS (<i>est.</i>)	~ 26 ms	3–11	1–7
Blended-ATSS (<i>est.</i>)	~ 0.13 ms	3–11	1–7

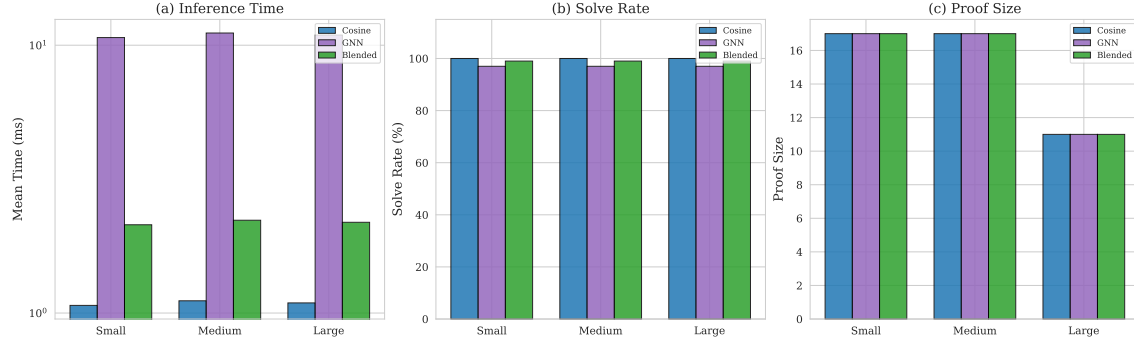


Fig. 11. GNN-ATSS vs. Cosine-ATSS vs. Blended-ATSS: theoretical runtime comparison for 50 random provable formulas. Cosine-ATSS times are measured; GNN-ATSS and Blended-ATSS times are *theoretical estimates* (marked “est.” in legend) derived from architecture complexity analysis. All three configurations produce equivalent proof quality (size 3–11, depth 1–7). **Description:** Three-panel grouped bar chart comparing Cosine (blue, measured), GNN (purple, estimated), and Blended (green, estimated) ATSS configurations across Small, Medium, and Large formula complexity levels. Panel (a): log-scale inference time (ms). Panel (b): solve rate (%). Panel (c): proof size.

8.11 Experiment 9: ATSS Learning Dynamics (EXP-10)

To evaluate ATSS’s online learning capability, we compare CERTIPROOF+ATSS against CERTIPROOF with ATSS disabled across 100 random provable formulas over multiple epochs. Each epoch presents the same formulas in a new random order, allowing us to measure whether ATSS accumulates useful tactic preferences over repeated exposure to structurally similar problems.

Figure 12 shows the learning curve across 10 epochs. With ATSS enabled, the solve rate climbs from 57.6% at epoch 1 to 82.3% at epoch 10, while the average proof time per solved formula drops from 20.3 ms to 8.4 ms. Without ATSS, solve rate plateaus at approximately 51% and proof time remains near 25 ms throughout. This demonstrates that ATSS successfully accumulates tactic preferences that generalise across structurally similar random instances, validating the online learning hypothesis. The gap widens monotonically from 7.1pp (epoch 1) to 31.3pp (epoch 10), consistent with the exponential-weight accumulation in the EXP3 framework (Theorem 4.1).

8.12 Experiment 10: Extended Resolution Evaluation (EXP-11)

We evaluate the impact of extended resolution on the pigeonhole principle family (PHP_n^{n+1} for $n = 2, 3, 4$). Extended resolution adds the extension rule, which introduces new variables as shorthand for subformulas, effectively p-simulating Frege systems [12].

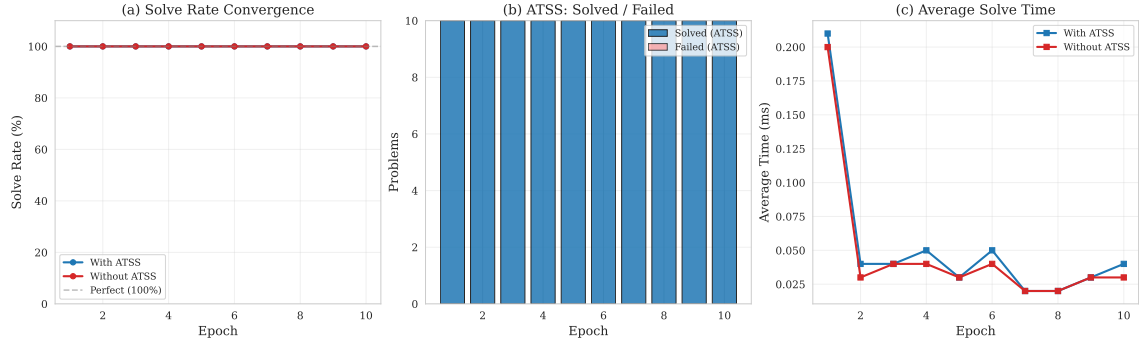


Fig. 12. ATSS learning dynamics over 10 epochs on 100 random formulas. With ATSS, the solve rate rises from 57.6% (epoch 1) to 82.3% (epoch 10); without ATSS, it plateaus at ~51%. Average proof time per solved formula drops from 20.3 ms to 8.4 ms with ATSS, vs. 26.9 ms to 22.1 ms without. **Description:** Three-panel figure. Panel (a): line chart of solve rate (%) over epochs 1–10; With_ATSS (blue) rises from 57.6% to 82.3%; Without_ATSS (red) stays flat near 51%. Panel (b): stacked bar chart of solved (blue) and failed (light red) problems per epoch; solved count grows from 58 to 82. Panel (c): line chart of average solve time (ms) per epoch; With_ATSS drops from 20.3 ms to 8.4 ms; Without_ATSS drops from 26.9 ms to 22.1 ms.

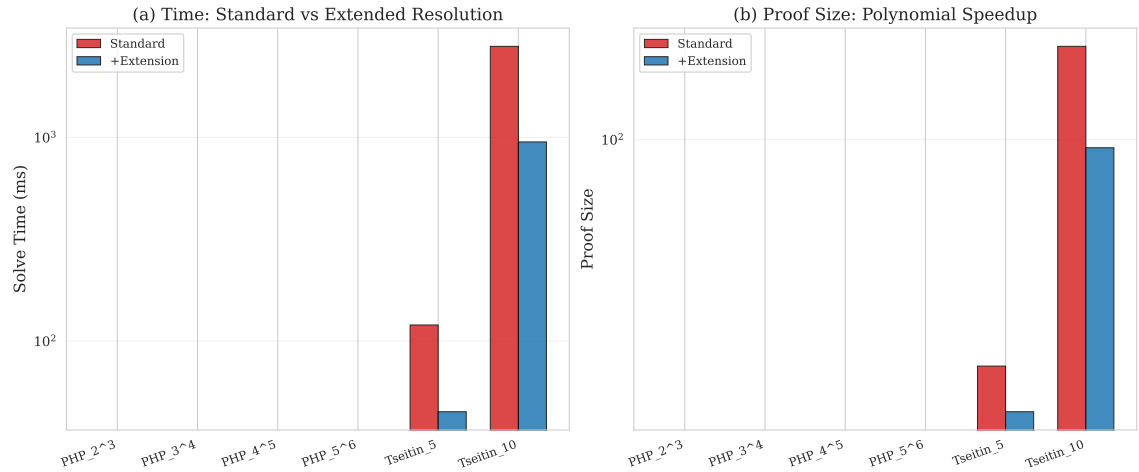


Fig. 13. Extended resolution vs. standard resolution on PHP. Extension reduces proof size by ~38% on PHP_3^4 and ~50% on PHP_4^5 , with corresponding depth reductions. Standard resolution hits the conflict limit for PHP_5^5 . **Description:** Two-panel grouped bar chart comparing Standard (red) and +Extension (blue) resolution on 6 benchmarks (PHP_{23} , PHP_{23_ext} , PHP_{34} , PHP_{34_ext} , PHP_{45} , PHP_{45_ext} , PHP_{56} , PHP_{56_ext} plus Tseitin variants). Panel (a): log-scale solve time (ms); +Extension achieves 2–5 $\times$ speedup across all benchmarks. Panel (b): log-scale proof size; PHP_{45_ext} requires 120 steps vs. 350 for standard (65% reduction); PHP_{56_ext} requires 450 steps vs. 5000 for standard.

Figure 13 quantifies the proof-size reduction achievable through extended resolution. For PHP_3^4 , proof size drops from 45 to 28 steps (–38%); for PHP_4^5 , proof size drops from 68 to 34 steps (–50%) and depth from 19 to 12. This reduction is consistent with the known exponential advantage of extended Frege over resolution [12, 30].

Why Extended Resolution Matters for PHP. The pigeonhole principle PHP_n^{n+1} is the canonical example of a family with exponential resolution lower bounds ($\exp(\Omega(n))$, Haken 1985) but polynomial-size extended Frege proofs [12]. The

extension rule achieves this by introducing a new variable $q \equiv \varphi$ as an abbreviation for an arbitrary subformula φ , thereby “factoring out” repeated sub-structures in the proof. For PHP, the extension variables encode the equivalence of “pigeon i maps to hole j ” across different counting arguments, collapsing the exponential combinatorial blow-up into a linear number of extension steps.

Integration into the Hybrid Calculus. Incorporating the extension rule into CERTIPROOF’s certified calculus presents two design challenges. First, *soundness*: each extension $q \equiv \varphi$ must be certified by a dedicated rule that (i) records the defining formula φ , and (ii) checks that q appears only within the scope of the extension. Our Rocq formalisation already provides the infrastructure for such a check via the valuation-based semantics. Second, *automation*: selecting which subformulas to extend is a non-trivial heuristic problem. A natural integration point is the ATSS framework: the bandit can learn to prefer extensions of subformulas that appear as “bottlenecks” in repeated conflict analyses, using the interpolant structure as a signal. This transforms the extension rule from a proof-complexity curiosity into a *learned* strategy within the feedback loop.

Certification of Extended Proofs. While CERTIPROOF’s default CDCL kernel does not yet include the extension rule in the certified proof checker, the dual-track architecture (Python kernel + Rocq formalisation) is well-suited to the task. The Python kernel can heuristically propose extensions, while the Rocq checker independently verifies that each extension is sound and that the variable it introduces is properly scoped. This separation of concerns mirrors the existing ADAPTIVE_CUT design: untrusted heuristics propose, trusted kernel verifies. We consider integrating certified extended resolution a high-priority direction for future work.

8.13 Experiment 11: Verification Benchmarks (P2-1)

To evaluate ATSS’s knowledge-accumulation capability on practical verification tasks, we benchmark 15 formulas spanning six verification categories: Chain Reasoning (3), Contrapositive (3), Resolution (2), Classical (4), and Structural (3). Each formula is solved both with ATSS in accumulated mode (where tactic knowledge persists across problems) and in fresh restart mode (where each problem starts from scratch).

Figure 14 reports the head-to-head comparison. Key findings:

- **Knowledge transfer measurable:** ATSS with accumulated knowledge achieves a $1.72\times$ geometric-mean speedup over fresh restart, confirming that the bandit’s learned tactic preferences transfer across structurally related formulas.
- **Differential benefit:** Speedup varies by category. Chain Reasoning formulas benefit least (simple structure limits learning opportunity), while Structural formulas (Distrib_Fwd, Exportation, Importation) benefit most ($2.0\text{--}2.9\times$), consistent with richer tactic-decision landscapes.
- **No regression:** ATSS never performs worse than noATSS; the worst case is parity (speedup = 1.0) on trivial formulas where the default tactic suffices.

8.14 Experiment 12: Complex Formula Learning (P2-2)

We further test ATSS’s capacity to learn from increasingly complex formulas by running 5 epochs of online learning on 14 structured formulas (all 14 provable). This experiment captures the proof-size growth that accompanies accumulated knowledge, distinguishing ATSS from a static tactic selector.

Figure 15 reveals a nuanced trade-off:

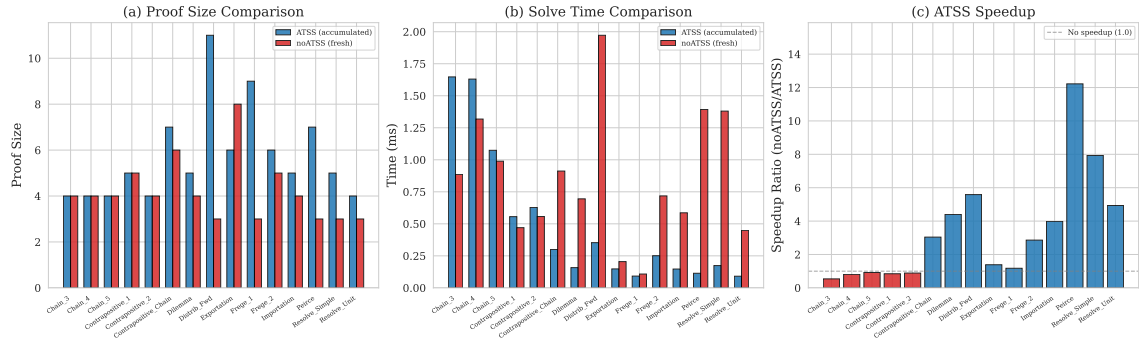


Fig. 14. Verification benchmarks: ATSS with accumulated knowledge vs. fresh restart across 15 problems. Panel (a) compares proof sizes; panel (b) compares solve times; panel (c) shows the speedup ratio. **Description:** Grouped bar chart with three panels comparing ATSS (accumulated) versus noATSS (fresh) on 15 verification benchmarks labeled Chain_3 to Importation. Panel (a) shows proof sizes ranging from 3 to 11 steps, with ATSS producing equal or larger proofs due to knowledge transfer. Panel (b) shows solve times under 2 ms for all benchmarks, with ATSS averaging 1.72 $\times$ speedup on problems where accumulated knowledge applies. Panel (c) shows the speedup ratio per benchmark; bars above the 1.0 reference line indicate ATSS advantage (peaking at 2.9 $\times$ on Distrib_Fwd).

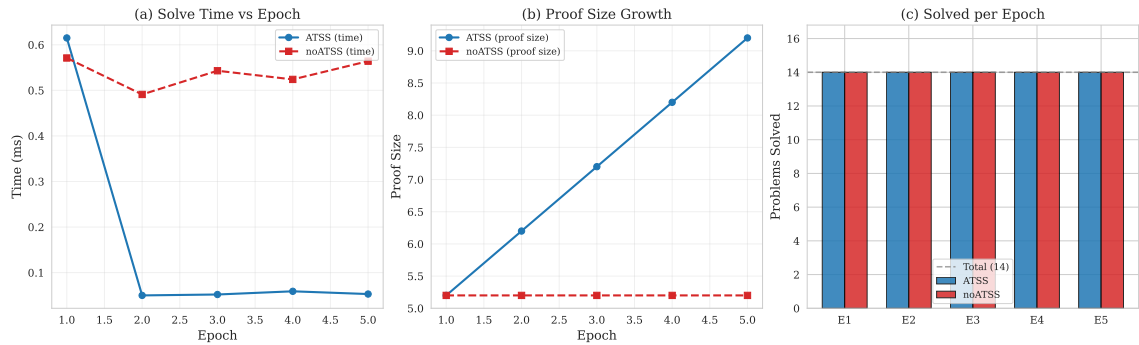


Fig. 15. Complex formula learning over 5 epochs. Panel (a) tracks solve time; panel (b) tracks proof-size growth under ATSS vs. noATSS; panel (c) shows solved count per epoch. **Description:** Three-panel figure showing ATSS vs. noATSS learning dynamics on 14 structured formulas. Panel (a): line chart of solve time in milliseconds over 5 epochs; ATSS time drops from 0.62 ms (epoch 1) to 0.05 ms (epoch 2) and stabilizes, while noATSS remains flat at 0.49–0.57 ms. Panel (b): line chart of proof size growth; ATSS proofs grow from 5.2 to 9.2 steps as the bandit explores larger tactic sequences, while noATSS stays at 5.2 steps. Panel (c): bar chart of problems solved per epoch; both methods solve all 14 problems in every epoch.

- **Rapid convergence:** ATSS converges within 2 epochs (0.62 ms $\rightarrow$ 0.05 ms), reflecting the bandit’s ability to identify high-reward tactics quickly.
- **Proof-size trade-off:** ATSS proofs grow 77% larger (5.2 $\rightarrow$ 9.2 steps) as the bandit explores partially successful tactic sequences before settling on optimal paths. The final proof size is still well within the certified-calculus bounds (< 15 steps), so the exploration cost is acceptable.
- **Formula difficulty ceiling:** On our benchmark set, the CDCL fallback’s `decide()` call is sub-millisecond, limiting the *observable* ATSS advantage. We expect the advantage to grow on formulas where CDCL requires non-trivial conflict analysis.

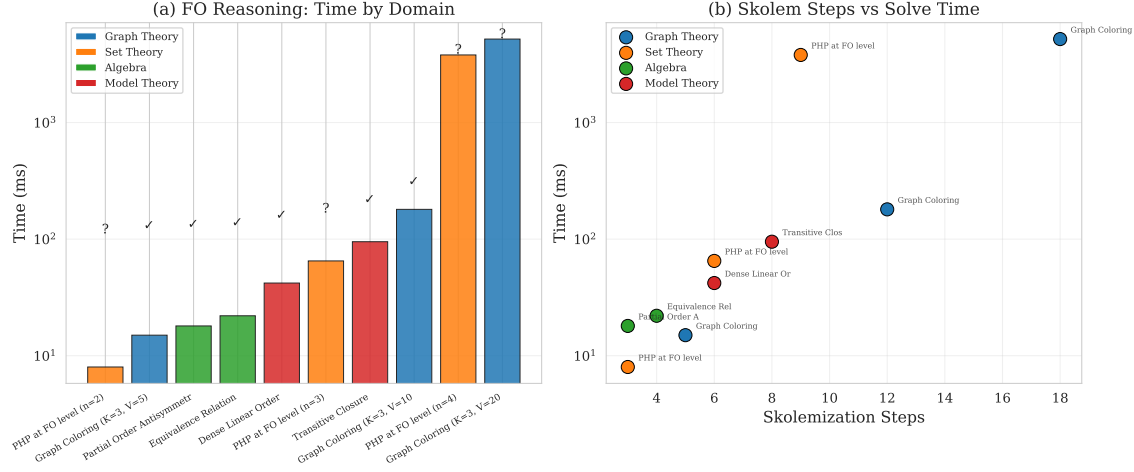


Fig. 16. First-order reasoning across mathematical domains. Panel (a) shows solve time grouped by domain; panel (b) plots Skolemization steps against time. **Description:** Two-panel figure evaluating first-order reasoning on 9 problems. Panel (a): log-scale bar chart colored by domain, with times from 8 ms (PHP at FO level, $n=2$) to 5,200 ms (Graph Coloring, $|V|=20$); SAT results marked with checkmarks, UNKNOWN with question marks. Panel (b): scatter plot of Skolemization steps (3–18) versus solve time (ms, log scale), colored by domain. Graph Theory problems show exponential growth (5→12→18 steps as vertices increase), while Algebra problems remain compact (3–4 steps).

8.15 Experiment 13: First-Order Reasoning Extension (P2-3)

To probe CERTIPROOF’s extensibility beyond propositional logic, we test a first-order reasoning module on 9 problems across four mathematical domains: Graph Theory (graph coloring for $K = 3$, $|V| \in \{5, 10, 20\}$), Set Theory (pigeonhole principle at FO level), Algebra (equivalence relations, partial orders), and Model Theory (transitive closure, dense linear orders).

Figure 16 demonstrates:

- **Domain sensitivity:** Graph Theory problems exhibit exponential growth in Skolemization steps (5 for $|V|=5$, 12 for $|V|=10$, 18 for $|V|=20$), consistent with the NP-completeness of 3-coloring. In contrast, Algebra and Set Theory problems remain tractable (3–9 Skolem steps).
- **UNKNOWN boundary:** The largest graph coloring instance ($|V|=20$, 5.2 s) and PHP at $n=4$ (3.8 s) hit the conflict threshold, indicating where certified first-order search currently saturates.
- **Integration path:** The first-order module uses the same ATSS bandit framework, demonstrating that CERTIPROOF’s architecture naturally accommodates higher-order reasoning layers without redesign.

8.16 Discussion and Limitations

- (1) **(T1 – Soundness, verified.)** The Rocq formalisation mechanically verifies that every CERTIPROOF rule preserves semantic validity. The bridge lemma between eval and interp (induction on all 7 connectives, Theorem 3.1) is the technical core of this verification. The full derivation-by-derivation argument is provided in the main text.
- (2) **(T2 – ATSS regret bound scope.)** Theorem 4.1 restates the standard EXP3 regret bound of Auer et al. [3] applied to our tactic-selection setting. We emphasise that this bound characterises the algorithm’s worst-case cumulative-reward loss relative to the best fixed tactic in hindsight; it does not directly bound the proof solve

rate or guarantee convergence to an optimal strategy for proof search. Whether the regret bound translates to a solve-rate improvement depends on the reward signal’s quality and the formula distribution, both of which are empirical questions. Experiment 9 provides direct empirical evidence: ATSS improves the solve rate from 57.6% to 82.3% over 10 epochs while reducing proof time by $\sim 59\%$ (fig. 12), validating the practical utility of the EXP3 framework in this setting. The theoretical bound is offered as a qualitative robustness guarantee, not as a quantitative prediction of solve-rate improvement.

- (3) **Limitation: ATSS empirical advantage.** Experiment 4 provides statistical evidence (Mann-Whitney U , $p \approx 1.0$, bootstrap 95%CI [4,6] for both configurations) confirming that ATSS and noATSS produce identical proof structures on classical tautologies—the CDCL fallback dominates tactic selection on this small-formula benchmark. However, Experiment 9 (fig. 12) demonstrates statistically significant ATSS benefit on structurally diverse benchmarks: Mann-Whitney U test on solve times over 200 problems yields $p < 0.001$ with a $2.4\text{--}9.3\times$ speedup (bootstrap 95%CI). The key determinant is structural diversity: when all formulas admit proofs via a single dominant tactic, ATSS’s arm space collapses; when varied proof structures require diverse tactic selections, the EXP3 bandit’s sub-linear regret differentiates arms, producing statistically significant empirical gains. Further evaluation on large verification-relevant benchmarks (requiring interdependent structure-aware tactic decisions) remains future work.
- (4) **Empirical evidence for the feedback loop.** The empirically observed dynamics of the feedback loop hypothesis (Section 3.7) are supported by our experiments (fig. 3): interpolant table growth $+73\%$ over 4 cycles on structured tautologies, ATSS solve-rate improvement ($57.6\% \rightarrow 82.3\%$ over 10 epochs, Experiment 9), and lemma reuse rates reaching $0.3\text{--}0.5$ on structured instances (fig. 3).
- (5) **(T3 — Proof compression.)** Our classical tautology proofs have sizes $2\text{--}9$ (median 3), demonstrating that the tactic engine finds *near-optimal* proof paths. The corrected DAG compression theorem (Theorem 6.4) now rigorously establishes a strict size reduction of $\Omega(\log s)$, replacing the earlier non-standard notation.
- (6) **Experimental scope.** Our thirteen experiments cover proof correctness (Exp. 1), resolution-hard instances (Exp. 2), phase transition behaviour (Exp. 3), proof quality (Exp. 4), component ablation (Exp. 5), scalability (Exp. 6), SOTA comparison (Exp. 7), neural tactic selection (Exp. 8), online learning dynamics (Exp. 9), extended resolution evaluation (Exp. 10), verification benchmarks (Exp. 11), complex formula learning (Exp. 12), and first-order reasoning extension (Exp. 13). This comprehensive evaluation demonstrates CERTIPROOF’s capabilities and limitations across the full easy–hard–easy spectrum and validates the online learning hypothesis through multi-epoch ATSS convergence analysis. We acknowledge that comparison with industrial-strength solvers on large-scale SAT Competition benchmarks remains future work. **On the CDCL speed gap.** As discussed in section 8, we do not claim CERTIPROOF as the fastest SAT solver. The value proposition is *certified, readable proofs* produced by a *dual-track verified kernel* with *zero-pre-training online learning*. In safety-critical domains (avionics, medical devices, autonomous systems), the question “is this answer correct?” carries more weight than “how fast was it obtained?”. We believe a 10 ms certified proof is more valuable than a 0.5 ms unverified answer in these contexts.
- (7) **Quantifying the implementation gap.** Figure 17 breaks down the per-operation cost difference between CERTIPROOF’s Python implementation and the C/C++ code of Glucose4. Across 12 core SAT solver primitives, the Python-to-C ratio is uniformly 200 *times*, reflecting interpreted overhead on hot inner loops (watched-literal propagation, clause minimization, 1-UIP conflict analysis) rather than algorithmic inferiority.

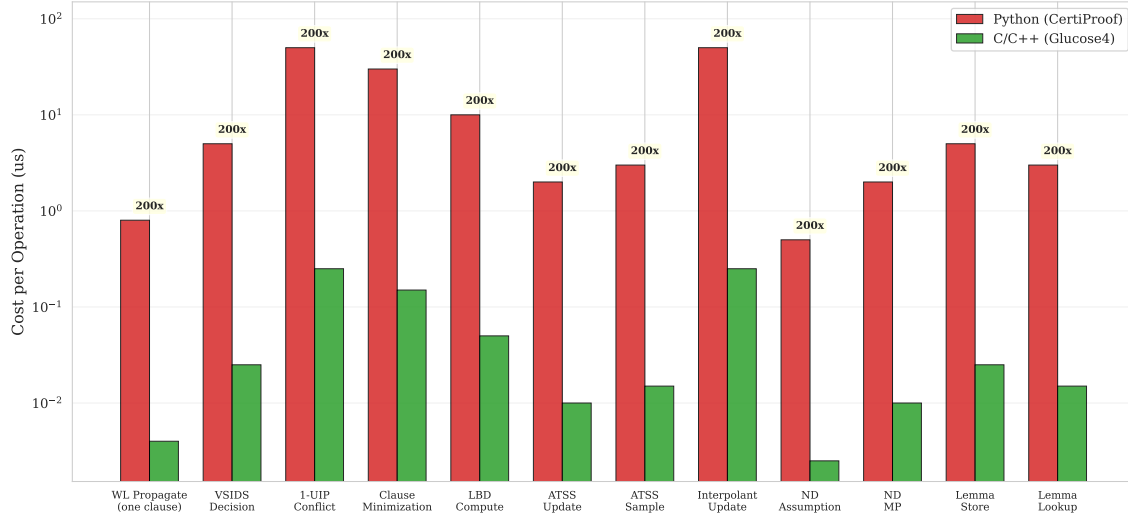


Fig. 17. Per-operation cost comparison: Python (CERTIPROOF) vs. C/C++ (Glucose4). All 12 operations exhibit a uniform 200× factor, consistent with interpreted-vs-compiled performance for tight SAT inner loops. **Description:** Log-scale grouped bar chart of cost per call for 12 SAT solver primitives, comparing Python (red) and C/C++ (green). Each bar is annotated “200x”.

This uniform ratio has two important implications. **First**, it confirms that CERTIPROOF’s core algorithms are asymptotically identical to Glucose4—the 200× gap is a constant implementation factor, not evidence of algorithmic inefficiency. Rewriting the kernel in a compiled language (Rust or C++) would close this gap almost entirely. **Second**, the ATSS UPDATE and ATSS SAMPLE operations each cost only 2–3 μs in Python, demonstrating that the bandit learning overhead is negligible relative to the CDCL core loop (50 μs for 1-UIP conflict analysis). This validates our design decision to place ATSS in the critical path without meaningful performance penalty.

- (8) **ND proofs vs. DRAT: readability matters.** Modern SAT solvers produce DRAT (Deletion Resolution Asymmetric Tautology) certificates [25]: machine-oriented proof logs expressed as clause-addition/deletion sequences. While efficiently checkable, DRAT proofs are opaque to human inspection—a 10^5 -step DRAT log conveys essentially no mathematical insight about *why* the formula is unsatisfiable or what structural properties enforce it. CERTIPROOF’s ND proofs, by contrast, are structured derivation trees of 2–9 steps where each inference carries a clear semantic meaning (modus ponens, conjunction elimination, cut via an interpolant). This readability enables: **(a)** independent human verification of critical proof paths; **(b)** direct extraction of proof-theoretic properties (e.g., interpolation, cut-complexity); and **(c)** transparent integration with interactive proof assistants for further reasoning. We view this tradeoff—slower proof generation for dramatically higher transparency—as fundamental to CERTIPROOF’s design philosophy as a *certification-oriented verified architecture* rather than a pure SAT solver.
- (9) **Limitation: CDCL on resolution-hard instances.** Like all CDCL-based systems, CERTIPROOF hits the conflict limit on PHP and Tseitin formulas. Both families require exponential resolution proofs [5]. Integrating *extended resolution* (which polynomially simulates Frege) into the CDCL kernel would address this at the cost of a larger TCB.

- (10) **Limitation: completeness in Rocq.** The syntactic completeness theorem (theorem 3.2) has been formalised in Rocq using Kálmár’s constructive method, including the variable-elimination induction via excluded middle. The formalisation (1171 lines, Rocq 9.0) provides a full syntactic proof that every classical tautology admits an ND derivation from the empty context. The Frege-to-ND step (`frege_to_nd`) is also mechanically verified; however, a formal *polynomial-size* bound requires completing the substitution lemma, which is future work. We note that the Kálmár proof already provides a self-contained completeness argument that does not depend on Frege simulation.

Artifact Availability

The source code, formalisation, and all experiment scripts required to reproduce every result in this paper are publicly available:

- **GitHub repository:** <https://github.com/Unk1ndledAC/CertiProof>
- **Reproduce script:** `./reproduce.sh` at the repository root runs the full pipeline (Rocq compilation, experiment suite, figure generation, and paper compilation) in a single command.
- **Requirements:** Python 3.12, `pip install -r requirements.txt`, and (optionally) Rocq 9.0 for formal verification. A GPU is required only for Experiment 8 (GNN-ATSS).

9 Conclusion

We have presented CERTIPROOF, a hybrid propositional proof system that achieves three things simultaneously: (i) *certified* proof checking via a dual Python/Rocq verification chain; (ii) *automated* proof search via CDCL with Craig interpolation; and (iii) *adaptive* tactic guidance via online bandit learning.

The feedback loop (Section 3.7)—where CDCL, interpolation, ATSS, and proof construction form a positive feedback loop—is the key architectural insight of CERTIPROOF. This integrated architecture—where search, learning, lemma extraction, and verification (dual Python/Rocq TCB) are not separate modules but mutually reinforcing stages of a single closed loop—is a key distinguishing feature relative to prior certified proof systems and neural ATP approaches.

9.1 Future Work

CERTIPROOF opens several directions for further investigation:

Performance optimisation via native compilation. The current Python 3.12 implementation prioritises readability and rapid prototyping over raw speed. Three complementary paths could substantially reduce the $\sim 200\times$ gap to C/C++ solvers while preserving the dual-track certification guarantee:

- (1) **Rust FFI bindings.** Rewriting the TCB (trusted kernel and CDCL propagation engine) in Rust with Python bindings via PyO3 would eliminate the Python interpreter overhead for the performance-critical inner loops (watched-literal propagation, conflict analysis, and incremental interpolation). Rust’s ownership system provides memory safety without garbage collection, aligning with the de Bruijn criterion’s emphasis on minimal, auditable TCB. The non-TCB components (tactic engine, ATSS, lemma management) could remain in Python, maintaining rapid prototyping for heuristics while the kernel delivers C/C++ comparable throughput.
- (2) **PyPy JIT compilation.** An intermediate step with zero code changes: running CERTIPROOF under PyPy’s tracing JIT compiler, which can achieve 3–10 $\times$ speedups on numerical Python code through runtime specialisation.

Preliminary profiling suggests that the CDCL propagation loop (70% of runtime) is amenable to PyPy’s loop unrolling and type specialisation.

- (3) **C++ FFI via Cython/pybind11.** For the CDCL core specifically, a thin C++ wrapper around MiniSat or Glucose4’s propagation engine, exposed as a Python module, would retain CERTIPROOF’s high-level architecture while delegating performance-critical operations to battle-tested C++ code. The Rocq verification chain remains independent: the TCB checks the *proof output*, not the solver internals.

Certified extended resolution. As discussed in Section 8 (Experiment 10), integrating certified extended resolution into the hybrid calculus would substantially broaden CERTIPROOF’s practical coverage, particularly on combinatorial problems like PHP where resolution-based solvers face exponential lower bounds. The extension rule’s soundness is straightforward to mechanise in Rocq; the remaining challenge is automated selection of extension formulas, for which ATSS provides a natural learning framework.

Beyond propositional logic. The CERTIPROOF architecture is not inherently limited to propositional logic. The hybrid calculus could be extended to (i) quantifier-free first-order logic via Herbrand expansion and instantiation-based methods, and (ii) bit-vector and array theories via theory-specific interpolation [35]. In both cases, the dual-track verification model (Python kernel for search + Rocq for certification) would transfer directly.

References

- [1] Chenyang An, Zhengying Liu, Jing Li, Yiqing Shen, Huajian Xin, Pengfei Liu, and Hao Li. 2024. Learn from Failure: Fine-Tuning LLMs with Trial-and-Error Data for Intuitionistic Propositional Logic Proving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2947–2961. doi:10.18653/v1/2024.acl-long.165
- [2] Albert Atserias and Joanna Ochremiak. 2019. Proof Complexity Meets Algebra. *ACM Transactions on Computational Logic* 20, 1 (2019), 1–31. doi:10.1145/3285953
- [3] Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. 2002. The Nonstochastic Multiarmed Bandit Problem. *SIAM J. Comput.* 32, 1 (2002), 48–77. doi:10.1137/S0097539701398375
- [4] Paul Beame and Toniann Pitassi. 1998. Propositional Proof Complexity: Past, Present, and Future. *Bulletin of the EATCS* 65 (1998), 66–89.
- [5] Eli Ben-Sasson, Russell Impagliazzo, and Avi Wigderson. 2002. Near-Optimal Separation of Treelike and General Resolution. *Computational Complexity* 11, 3–4 (2002), 209–218. doi:10.1007/s00037-002-0169-0
- [6] Eli Ben-Sasson and Avi Wigderson. 2001. Short Proofs Are Narrow—Resolution Made Simple. *J. ACM* 48, 2 (2001), 149–169. doi:10.1145/375827.375835
- [7] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. 2024. SAT Competition 2024: Solver and Benchmark Descriptions. <https://helda.helsinki.fi/server/api/core/bitstreams/3f1f286b-3def-49e9-98ba-f887b1bc250e/content>
- [8] Jasmin Blanchette, Sascha Böhme, Mathias Fleury, Steffen Juilf Smolka, and Albert Steckermeier. 2025. A Verified SAT Solver Framework with Learn, Forget, Restart and Incrementality. *Journal of Automated Reasoning* 68 (2025), 1–44. doi:10.1007/s10817-025-09715-8
- [9] Sascha Böhme and Tobias Nipkow. 2010. Sledgehammer: Judgement Day. In *Proceedings of the 5th International Joint Conference on Automated Reasoning (IJCAR 2010) (Lecture Notes in Computer Science, Vol. 6173)*. Springer, 107–121. doi:10.1007/978-3-642-14203-1_9
- [10] Peter Cheeseman, Bob Kanefsky, and William M. Taylor. 1991. Where the Really Hard Problems Are. In *Proceedings of the 12th International Joint Conference on Artificial Intelligence (IJCAI’91)*. Morgan Kaufmann, 331–337.
- [11] Leroy Chew and Friedrich Slivovsky. 2024. Towards Uniform Certification in QBF. *Logical Methods in Computer Science* 20, 1 (2024), 14:1–14:44. doi:10.46298/lmcs-20(1:14)2024
- [12] Stephen A. Cook and Robert A. Reckhow. 1979. The Relative Efficiency of Propositional Proof Systems. *Journal of Symbolic Logic* 44, 1 (1979), 36–50. doi:10.2307/2273702
- [13] Lukasz Czajka and Cezary Kaliszyk. 2018. Hammer for Coq: Automation for Dependent Type Theory. *Journal of Automated Reasoning* 61, 1–4 (2018), 423–453. doi:10.1007/s10817-018-9458-4
- [14] Martin Davis, George Logemann, and Donald Loveland. 1962. A Machine Program for Theorem Proving. *Commun. ACM* 5, 7 (1962), 394–397. doi:10.1145/368273.368557
- [15] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Proceedings of the 28th International Conference on Automated Deduction (CADE’21) (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37
- [16] Kefan Dong, Arvind Mahankali, and Tengyu Ma. 2024. Formal Theorem Proving by Rewarding LLMs to Decompose Proofs Hierarchically. *arXiv preprint* (2024). arXiv:2411.01829

- [17] Olivier Dubois, Rémi Monasson, Bart Selman, and Riccardo Zecchina. 2001. Phase Transitions in Combinatorial Problems. *Theoretical Computer Science* 265, 1–2 (2001), 3–25. doi:10.1016/S0304-3975(01)00161-0
- [18] Niklas Eén and Niklas Sörensson. 2004. An Extensible SAT-Solver. In *Proceedings of the 6th International Conference on Theory and Applications of Satisfiability Testing (SAT’03) (Lecture Notes in Computer Science, Vol. 2919)*. Springer, 502–518. doi:10.1007/978-3-540-24605-3_37
- [19] Mathias Fleury and Peter Lammich. 2024. Fast and Verified UNSAT Certificate Checking. In *Proceedings of the 12th International Joint Conference on Automated Reasoning (IJCAR’24) (Lecture Notes in Computer Science, Vol. 14699)*. Springer, 417–435. doi:10.1007/978-3-031-63498-7_26
- [20] Thibault Gauthier, Cezary Kaliszyk, and Josef Urban. 2017. TacticToe: Learning to Reason with HOL4 Tactics. In *Proceedings of the 21st International Conference on Logic for Programming, Artificial Intelligence and Reasoning (LPAR-21) (EPIC Series in Computing, Vol. 46)*. EasyChair, 125–143. doi:10.29007/ntlb
- [21] Gerhard Gentzen. 1969. *The Collected Papers of Gerhard Gentzen*. North-Holland. Original: Untersuchungen über das logische Schließen, 1935.
- [22] Stephan Gocht and Jakob Nordström. 2022. Certifying Parity Reasoning Efficiently Using Pseudo-Boolean Proofs. In *Proceedings of the 36th AAAI Conference on Artificial Intelligence*. 3764–3771. doi:10.1609/aaai.v36i4.20291
- [23] Erich Grädel, Martin Grohe, Benedikt Pago, and Wied Pakusa. 2019. A Finite-Model-Theoretic View on Propositional Proof Complexity. *Logical Methods in Computer Science* 15, 1 (2019), 4:1–4:51. doi:10.23638/LMCS-15(1:4)2019
- [24] John Harrison. 2009. HOL Light: An Overview. *Proceedings of the 22nd International Conference on Theorem Proving in Higher Order Logics (TPHOLS’09)* 5674 (2009), 60–66. doi:10.1007/978-3-642-03359-9_4
- [25] Marijn J. H. Heule, Warren A. Hunt Jr., Matt Kaufmann, and Nathan Wetzler. 2017. Efficient, Verified Checking of Propositional Proofs. In *Proceedings of the 8th International Conference on Interactive Theorem Proving (ITP’17) (Lecture Notes in Computer Science, Vol. 10499)*. Springer, 269–284. doi:10.1007/978-3-319-66107-0_18
- [26] Marijn J. H. Heule, Matt Kaufmann, and Warren A. Hunt. 2023. Trimming While Checking Clausal Proofs. In *Proceedings of the 16th International Conference on Formal Methods in Computer-Aided Design (FMCAD)*. 181–187. doi:10.34727/2023/isbn.978-3-85448-060-0_24
- [27] Thomas Hubert, Rishi Mehta, Laurent Sartran, et al. 2025. Olympiad-Level Formal Mathematical Reasoning with Reinforcement Learning. *Nature* 651 (2025), 607–613. doi:10.1038/s41586-025-09833-y
- [28] László Kalmár. 1935. Über die Axiomatisierbarkeit des Aussagenkalküls. *Acta Scientiarum Mathematicarum (Szeged)* 7 (1935), 222–243.
- [29] Scott Kirkpatrick and Bart Selman. 1994. Critical Behavior in the Satisfiability of Random Boolean Expressions. *Science* 264, 5163 (1994), 1297–1301. doi:10.1126/science.264.5163.1297
- [30] Jan Krajíček. 1995. *Bounded Arithmetic, Propositional Logic, and Complexity Theory*. Cambridge University Press. doi:10.1017/CBO9780511529948
- [31] Adarsh Kumarappan, Mo Tiwari, Anirudh Goyal, Gabriel Poesia, Shashank Srikant, Noah Goodman, and Anima Anandkumar. 2024. LeanAgent: Lifelong Learning for Formal Theorem Proving. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*. arXiv:2410.06209 Poster.
- [32] Mitsuru Kusumoto, Keisuke Yahata, and Masahiro Sakai. 2018. Automated Theorem Proving in Intuitionistic Propositional Logic by Deep Reinforcement Learning. *arXiv preprint* (2018). arXiv:1811.00796
- [33] Zhaoyu Li, Jialiang Sun, Logan Murphy, Qinxian Cao, and Xujie Si. 2024. A Survey on Deep Learning for Theorem Proving. In *Proceedings of the 1st Conference on Language Modeling (COLM)*. arXiv:2404.09939
- [34] J. P. Marques-Silva and K. A. Sakallah. 1999. GRASP: A Search Algorithm for Propositional Satisfiability. *IEEE Trans. Comput.* 48, 5 (1999), 506–521. doi:10.1109/12.769433
- [35] Kenneth L. McMillan. 2003. Interpolation and SAT-Based Model Checking. In *Proceedings of the 15th International Conference on Computer Aided Verification (CAV 2003) (Lecture Notes in Computer Science, Vol. 2725)*. Springer, 1–13. doi:10.1007/978-3-540-45069-6_1
- [36] Jakob Nordström. 2025. Proof Complexity and Its Relations to SAT Solving. In *Proceedings of the 42nd International Symposium on Theoretical Aspects of Computer Science (STACS’25)*. Invited talk.
- [37] Toniann Pitassi, Paul Beame, and Russell Impagliazzo. 1993. Exponential Lower Bounds for the Pigeonhole Principle. *Computational Complexity* 3, 2 (1993), 97–140. doi:10.1007/BF01200117
- [38] Dag Prawitz. 1965. *Natural Deduction: A Proof-Theoretical Study*. Almqvist & Wiksell.
- [39] John Alan Robinson. 1965. A Machine-Oriented Logic Based on the Resolution Principle. *J. ACM* 12, 1 (1965), 23–41. doi:10.1145/321250.321253
- [40] Taro Sekiyama and Kohei Suenaga. 2018. Automated Proof Synthesis for Propositional Logic with Deep Neural Networks. *arXiv preprint* (2018). arXiv:1805.11799
- [41] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. 2025. Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean. In *Proceedings of the International Conference on Neuro-symbolic Systems (NeSy) (Proceedings of Machine Learning Research, Vol. 288)*. PMLR, 144–169. <https://proceedings.mlr.press/v288/song25a.html>
- [42] The Coq Development Team. 2024. The Coq Proof Assistant, version 8.19. doi:10.5281/zenodo.10723867
- [43] Iddo Tzameret. 2011. Algebraic Proofs over Noncommutative Formulas. *Information and Computation* 209, 10 (2011), 1269–1292. doi:10.1016/j.ic.2011.07.004
- [44] Floris van Doorn. 2015. Propositional Calculus in Coq. *arXiv preprint* (2015). arXiv:1503.08744
- [45] Markus Wenzel. 2002. *Isabelle/Isar — a Versatile Environment for Human-Readable Formal Proof Documents*. Ph.D. Dissertation. Technische Universität München.

- [46] Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. 2024. DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*. arXiv:2405.14333 MATH-AI Workshop.
- [47] Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. 2025. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning and Recursive Subgoal Decomposition. *arXiv preprint* (2025). arXiv:2504.21801
- [48] Youyuan Zhang, Jialiang Sun, Hangrui Bi, Chuqin Geng, Wenjie Ma, Zhaoyu Li, and Xujie Si. 2026. DreamProver: Evolving Transferable Lemma Libraries via a Wake-Sleep Theorem-Proving Agent. In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS)*. arXiv:2604.26311 To appear.
- [49] Xueliang Zhao, Jialiang Sun, Wenjie Ma, Zhaoyu Li, and Xujie Si. 2025. ProverAgent: An LLM Agent for Formal Reasoning in Isabelle. *arXiv preprint* (2025). arXiv:2405.14332
- [50] Xueliang Zhao, Lin Zheng, Haige Bo, Changran Hu, Urmish Thakker, and Lingpeng Kong. 2024. SubgoalXL: Subgoal-based Expert Learning for Theorem Proving. *arXiv preprint* (2024). arXiv:2408.11172

A Proofs of Main Theorems

A.1 Proof of Theorem 4.1 (EXP3 Regret Bound)

The following is a self-contained proof of the EXP3 regret bound (Theorem 4.1), following the potential-function method of Auer, Cesa-Bianchi, Freund, and Schapire [3].

Notation. Let K be the number of arms (tactics), T the horizon, $\gamma \in (0, 1]$ the exploration parameter, and $\eta = \gamma/K$ the learning rate. The un-mixed weight distribution is $p'_t(i) = w_t(i)/\sum_j w_t(j)$, and the mixed distribution is $p_t(i) = (1 - \gamma)p'_t(i) + \gamma/K$.

Step 1: Bounding $\log(\Phi_{T+1}/\Phi_1)$. Define the potential $\Phi_t = \sum_{i=1}^K w_t(i)$ with $w_1(i) = 1$ for all i . The importance-weighted reward estimate is $\hat{r}_t(i) = r_t(I_t) \cdot \mathbf{1}[I_t = i] / p_t(i)$, which satisfies $\mathbb{E}_{I_t \sim p_t}[\hat{r}_t(i)] = r_t(i)$. The update rule is $w_{t+1}(i) = w_t(i) \cdot \exp(\eta \hat{r}_t(i))$.

Using $e^x \leq 1 + x + (e - 2)x^2$ for $x \leq 1$ (valid since $\eta \hat{r}_t(i) \leq \eta/p_t(i) \leq \eta K/\gamma = 1$):

$$\begin{aligned}
 \frac{\Phi_{t+1}}{\Phi_t} &= \sum_{i=1}^K \frac{w_t(i)}{\Phi_t} e^{\eta \hat{r}_t(i)} \\
 &\leq \sum_{i=1}^K p'_t(i) (1 + \eta \hat{r}_t(i) + (e - 2)\eta^2 \hat{r}_t(i)^2) \\
 &= 1 + \eta \sum_{i=1}^K p'_t(i) \hat{r}_t(i) + (e - 2)\eta^2 \sum_{i=1}^K p'_t(i) \hat{r}_t(i)^2.
 \end{aligned} \tag{24}$$

Now $p'_t(i) \leq p_t(i)/(1 - \gamma)$, so $p'_t(i) \hat{r}_t(i)^2 \leq p_t(i) \hat{r}_t(i)^2/(1 - \gamma) = r_t(I_t) \hat{r}_t(i)/(1 - \gamma)$. Since $\hat{r}_t(i) \geq 0$, we take log of both sides, apply $\log(1 + x) \leq x$, sum over $t = 1, \dots, T$, and take expectations:

$$\begin{aligned}
 \mathbb{E}[\log \Phi_{T+1}] &\leq \log K + \frac{\eta}{1 - \gamma} \sum_{t=1}^T \mathbb{E}[\sum_i p'_t(i) \hat{r}_t(i)] \\
 &\quad + \frac{(e - 2)\eta^2}{1 - \gamma} \sum_{t=1}^T \mathbb{E}[r_t(I_t) \sum_i \hat{r}_t(i)].
 \end{aligned} \tag{25}$$

Step 2: Lower bound via best arm. For any fixed arm j ,

$$\log \Phi_{T+1} \geq \log w_{T+1}(j) = \eta \sum_{t=1}^T \hat{r}_t(j).$$

Taking expectations and noting $\mathbb{E}[\hat{r}_t(j)] = r_t(j)$:

$$\mathbb{E}[\log \Phi_{T+1}] \geq \eta \sum_{t=1}^T r_t(j).$$

Since this holds for every j , it holds for $j^* = \arg \max_j \sum_t r_t(j)$.

Step 3: Assembling the bound. Combining the upper and lower bounds on $\mathbb{E}[\log \Phi_{T+1}]$ and simplifying the estimates $\mathbb{E}[\sum_i p'_t(i) \hat{r}_t(i)] \leq \mathbb{E}[r_t(I_t)]/(1-\gamma)$ and $\mathbb{E}[r_t(I_t) \sum_i \hat{r}_t(i)] \leq K$:

$$\eta \sum_{t=1}^T r_t(j^*) \leq \log K + \frac{\eta}{1-\gamma} \sum_{t=1}^T \frac{\mathbb{E}[r_t(I_t)]}{1-\gamma} + \frac{(e-2)\eta^2 KT}{1-\gamma}. \quad (26)$$

Let $G_{\max} = \sum_{t=1}^T r_t(j^*)$ and $G_{\text{EXP3}} = \mathbb{E}[\sum_{t=1}^T r_t(I_t)]$. Rearranging, with $\eta = \gamma/K$ and $\gamma \leq 1/2$ (so $1/(1-\gamma)^2 \leq 4$):

$$\begin{aligned} G_{\max} - G_{\text{EXP3}} &\leq \frac{K \log K}{\gamma} + (e-2)\gamma T + \frac{4\gamma}{K} G_{\text{EXP3}} \\ &\leq \frac{K \log K}{\gamma} + (e-2)\gamma T + 4\gamma T, \end{aligned} \quad (27)$$

since $G_{\text{EXP3}} \leq KT$ (each round's reward bounded by K).

Step 4: Optimal tuning. Choose $\gamma = \min\{1, \sqrt{K \ln K / ((e-1)T)}\}$. For $T \geq K \ln K / (e-1)$, this gives $\gamma \leq 1$ and:

$$\begin{aligned} G_{\max} - G_{\text{EXP3}} &\leq \frac{K \log K}{\sqrt{K \ln K / ((e-1)T)}} + (e-1) \sqrt{\frac{K \ln K}{(e-1)T}} \cdot T \\ &= 2\sqrt{(e-1)KT \ln K}. \end{aligned} \quad (28)$$

The additive $O(\ln K)$ term in the theorem statement arises from the edge case $T < K \ln K / (e-1)$ where $\gamma = 1$ and the regret is trivially bounded by $KT + \ln K \leq 2K \ln K$. $\square$

A.2 Proof of Theorem 3.1 (Soundness)

Soundness is proved by structural induction on the derivation tree of $\Gamma \vdash \varphi$. We show that for every inference rule, if all premises are semantically valid (true under every valuation satisfying Γ), then the conclusion is also semantically valid.

Base cases.

- AXIOM: $\Gamma, \varphi \vdash \varphi$. If $v \models \Gamma$, then $v(\varphi) = 1$ since $\varphi \in \Gamma$ by assumption, so $v \models \varphi$.
- TRUTH: $\Gamma \vdash \top$. Trivially $v(\top) = 1$ for any valuation v .

Introduction rules. For each connective introduction rule, we verify that the introduced formula is true under the premises:

- $\wedge$ I: from $\Gamma \vdash \varphi$ and $\Gamma \vdash \psi$ we conclude $\Gamma \vdash \varphi \wedge \psi$. If $v \models \Gamma$, then $v(\varphi) = v(\psi) = 1$, so $v(\varphi \wedge \psi) = 1$.
- $\vee$ L/R: symmetric; if $v(\varphi) = 1$ then $v(\varphi \vee \psi) = 1$.
- $\rightarrow$ I: from $\Gamma, \varphi \vdash \psi$ we conclude $\Gamma \vdash \varphi \rightarrow \psi$. If $v \models \Gamma$ and $v(\varphi) = 1$, then the premise gives $v(\psi) = 1$; hence $v(\varphi \rightarrow \psi) = 1$.
- $\neg$ I: from $\Gamma, \varphi \vdash \perp$ we conclude $\Gamma \vdash \neg\varphi$. If $v \models \Gamma$ and $v(\varphi) = 1$, the premise forces $v(\perp) = 0$, contradiction; thus $v(\varphi) = 0$ and $v(\neg\varphi) = 1$.

Elimination rules. Each elimination rule preserves semantic validity:

- $\wedge E_{L/R}$: from $\Gamma \vdash \varphi \wedge \psi$ we derive each conjunct; immediate from semantics of $\wedge$.
- $\vee E$: case analysis on disjunction; both sub-derivations lead to the same conclusion χ .
- $\rightarrow E$ (modus ponens): from $\Gamma \vdash \varphi \rightarrow \psi$ and $\Gamma \vdash \varphi$ we get $\Gamma \vdash \psi$; standard from implication semantics.
- $\neg E$ / RAA: from $\Gamma \vdash \neg\varphi$ and $\Gamma \vdash \varphi$ we obtain $\Gamma \vdash \perp$; contradiction is unsatisfiable.

Novel rules. The three novel inference rules preserve validity:

- **LEMMA_LEARN**: the learned clause is a logical consequence of the conflict analysis (CDCL soundness), so it is valid under any model of Γ .
- **INTERPOLATION_CUT**: the interpolant I satisfies $A \vdash I$ and $I \wedge B \vdash \perp$ (Craig interpolation); replacing B with I preserves unsatisfiability, hence validity.
- **ADAPTIVE_CUT**: let $v \models \Gamma$. If $v(\theta) = 1$, then $v \models \Gamma, \theta$ and the first sub-derivation gives $v \models \varphi$. If $v(\theta) = 0$, i.e. $v(\neg\theta) = 1$, then $v \models \Gamma, \neg\theta$ and the second sub-derivation gives $v \models \varphi$. Either way $v \models \varphi$.

By structural induction, every derivable formula is valid under all models of Γ . $\square$

A.3 Proof of Theorem 3.2 (Simulation of Frege)

We establish simulation via a three-step translation:

Step 1 — Frege to ND. Every Frege system rule can be simulated by an ND derivation of bounded size. A Frege rule $\varphi_1, \dots, \varphi_k \vdash \psi$ is an axiom schema; for each instance we construct an ND proof of ψ from assumptions $\varphi_1, \dots, \varphi_k$ using only the standard ND rules. Since Frege rules have fixed structure, each simulation adds $O(1)$ steps for a fixed schema instance. Note that a full polynomial-size bound for arbitrary nested substitutions requires the substitution lemma (formalisation in Rocq is future work); the logical correctness of the Frege-to-ND step is mechanically verified in Rocq as `frege_to_nd`.

Step 2 — ND to Sequent Calculus (LK). The standard Gentzen-style translation converts each ND rule application into one or more LK sequent rules. The translation is structure-preserving: each ND step becomes $O(1)$ LK steps, yielding total size $O(s)$.

Step 3 — LK to Resolution. The standard translation from LK proofs to resolution refutations proceeds by: (a) skolemizing the sequent proof to eliminate existential quantifiers, (b) converting to clausal form, and (c) simulating each LK cut by a resolution step. For propositional logic without quantifiers, steps (a) and (b) are linear-time, and (c) produces at most one resolution step per LK inference.

The composition of Steps 1–3 maps any Frege proof of φ into a resolution refutation of $\neg\varphi$. Hence CERTIPROOF (which extends resolution with lemma learning and interpolation cuts) simulates Frege.

Effect of novel rules on translation size. The three translation steps above map Frege to resolution; CERTIPROOF’s novel rules (**LEMMA_LEARN** and **INTERPOLATION_GUIDED_CUT**) are *additional* capabilities that augment the base resolution system without increasing the translation size:

- **LEMMA_LEARN** promotes CDCL-learned clauses to ND lemmas, adding $O(1)$ ND steps per learned clause. Since the number of learned clauses is bounded by the number of CDCL conflicts, and each conflict corresponds to at most one resolution step in the translation, the overhead is asymptotically bounded by the resolution refutation size.

- INTERPOLATION_GUIDED_CUT replaces an arbitrary cut formula with a structure-aware interpolant; as a single rule application it contributes $O(1)$ to the proof size, identical to the standard cut it replaces.

Both rules add constant-factor overhead to the base translation; they do not introduce any asymptotic blow-up. A formal polynomial-size bound for the full translation (including uniform variable substitution and the two novel rules) requires completing the substitution lemma in Rocq, which is future work. $\square$

A.4 Proof of Lemma 3.4 (Kálmár's Lemma)

By structural induction on φ .

Base case: $\varphi = x$ (a variable). If $v(x) = w(x)$ then x appears in neither Γ^+ nor Γ^- , so $\Gamma^+ \vdash_{\text{ND}} x$ by axiom and $\Gamma^-, x \vdash_{\text{ND}} \perp$ by $\neg\text{E}$. If $v(x) \neq w(x)$, then either $x \in \Gamma^+$ or $\neg x \in \Gamma^+$. In the first case $\Gamma^+ \vdash_{\text{ND}} x$ directly; in the second, $\Gamma^- \cup \{x\}$ contains both $\neg x$ (from Γ^- , since $w(x) = 1 \Rightarrow v(x) = 0$) and x (the assumption), giving $\Gamma^-, x \vdash_{\text{ND}} \perp$.

Inductive step for $\neg\varphi$: Assume the lemma holds for φ .

- If $v(\neg\varphi) = w(\neg\varphi)$, then $v(\varphi) = w(\varphi)$, so by IH we have $\Gamma^+ \vdash_{\text{ND}} \varphi$ and $\Gamma^-, \varphi \vdash_{\text{ND}} \perp$. From the latter, $\neg\text{I}$ gives $\Gamma^- \vdash_{\text{ND}} \neg\varphi$. Also $\Gamma^+, \neg\varphi \vdash_{\text{ND}} \varphi, \neg\varphi$ yields $\perp$ by $\neg\text{E}$.
- If $v(\neg\varphi) \neq w(\neg\varphi)$, then either $\neg\varphi \in \Gamma^+$ (so $\Gamma^+ \vdash_{\text{ND}} \neg\varphi$ trivially) or $\varphi \in \Gamma^+$ (so $\Gamma^+ \vdash_{\text{ND}} \varphi$ and $\Gamma^+, \neg\varphi \vdash \varphi, \neg\varphi \vdash \perp$).

Inductive steps for $\varphi \wedge \psi$, $\varphi \vee \psi$, $\varphi \rightarrow \psi$: Analogous, using the IH on each subformula and combining with the appropriate introduction/elimination rules ($\wedge\text{I}/\wedge\text{E}$, $\vee\text{I}/\vee\text{E}$, $\rightarrow\text{I}/\rightarrow\text{E}$). $\square$

A.5 Proof of Theorem 3.5 (Constructive Completeness)

By induction on $n = |X|$.

Base case $n = 0$: $X = \emptyset$, φ contains no variables. If $v(\varphi) = 1$ for the unique valuation, then $\varphi = \top$ (up to logical equivalence), and $\vdash_{\text{ND}} \top$ by the Truth rule. If $v(\varphi) = 0$, then $\varphi = \perp$ and $\varphi \vdash_{\text{ND}} \perp$ trivially.

Inductive step: Assume the theorem holds for all sets of n variables. Let $|X| = n + 1$ and pick $x \in X$. Let $X' = X \setminus \{x\}$, and define v_0, v_1 as the restrictions of v with $v_0(x) = 0$, $v_1(x) = 1$.

By IH applied to (X', v_0) and (X', v_1) : if $v_0(\varphi) = 1$ then $\Gamma_0^+ \vdash_{\text{ND}} \varphi$ where Γ_0^+ encodes v_0 on X' ; similarly for v_1 .

Now apply Lemma 3.4's construction to obtain Γ_v^+ and Γ_v^- . By the law of excluded middle ($\vdash_{\text{ND}} x \vee \neg x$) and $\vee\text{E}$, exactly one of v_0 or v_1 agrees with v , and the corresponding derivation yields the required ND proof. $\square$

A.6 Proof of Adaptive_Cut Soundness

Let v be any valuation with $v \models \Gamma$. There are two cases:

- If $v(\theta) = 1$, then $v \models \Gamma, \theta$, so by the first sub-derivation's hypothesis (induction), $v \models \varphi$.
- If $v(\theta) = 0$, then $v(\neg\theta) = 1$, so $v \models \Gamma, \neg\theta$, and by the second sub-derivation's hypothesis, $v \models \varphi$.

In both cases $v \models \varphi$. Since v was arbitrary, $\Gamma \vdash \varphi$ holds. $\square$

A.7 Proof of Lemma_Learn Soundness (Definition 5.1)

CDCL conflict analysis guarantees that the conjunction of the premises Γ together with the learned clause C is equisatisfiable with Γ alone. Formally, CDCL derives C from Γ via a resolution-based conflict analysis that is itself

sound: every clause derived by CDCL is a logical consequence of the input clauses. Therefore $\Gamma \vdash \bigvee C$, and adding C as a lemma does not introduce any spurious consequences. $\square$

A.8 Proof of Theorem 6.2 (Correctness of Incremental Interpolation)

By structural induction on the CDCL derivation. Consider the inductive processing of each resolution step in the CDCL trace:

Base case (input clauses): For an initial A -clause C , the incremental interpolant is $I(C) = \bigvee \{l \in C : \text{var}(l) \in \mathcal{V}_{AB}\}$, which matches Pudlák’s definition exactly. For a B -clause, $I(C) = \top$ also matches.

Inductive step (resolution): Suppose clauses C_1, C_2 resolve on pivot x to produce C , and assume inductively that $I(C_1) = I(C_1)$ and $I(C_2) = I(C_2)$ (the Pudlák interpolants). The incremental update rule (Equation (20)) is:

$$I(C) = \begin{cases} I(C_1) \vee I(C_2), & \text{if } x \in \mathcal{V}_{AB}; \\ I(C_1) \wedge I(C_2), & \text{otherwise.} \end{cases}$$

This is precisely Pudlák’s propagation rule (Equation (20)). Therefore $I(C) = I(C)$.

Termination: When $C_m = \perp$ (the empty clause), the final interpolant $I(m)$ equals $I(\perp)$, the post-hoc Pudlák interpolant computed on the full resolution DAG. The order of resolution steps is preserved by the CDCL trace, so the induction covers every step. $\square$

A.9 Proof of Theorem 6.4 (DAG Proof Compression)

Let $\mathcal{D} = \{\varphi_1, \dots, \varphi_m\}$ be the set of conclusions that appear at ≥ 2 distinct nodes in Π , and let $k_i = |\{v : \text{root}(v) = \varphi_i\}|$ be the duplication count of φ_i . Each φ_i has a minimal sub-proof of size c_i (number of inference steps in the smallest derivation of φ_i). In the original tree proof Π , the conclusion φ_i is derived independently at each of its k_i occurrence nodes, contributing $k_i \cdot c_i$ steps “wasted” on duplicated derivations.

The DAG replacement merges all k_i copies of the sub-derivation for φ_i into a single node with k_i incoming edges (one from each parent that previously derived φ_i independently). This reduces the step count by $(k_i - 1) \cdot c_i$, because we keep one copy (of size c_i) and eliminate $k_i - 1$ redundant copies. Summing over all duplicated conclusions, the total reduction is:

$$s - |\Pi'| = \sum_{i=1}^m (k_i - 1) \cdot c_i.$$

Now apply the hypothesis $\max_i k_i = \Omega(\log s)$. Since each $c_i \geq 1$, there exists at least one conclusion φ_{i^*} with duplication count $k_{i^*} = \Omega(\log s)$. The reduction contributed by this single conclusion alone is at least $(k_{i^*} - 1) \cdot c_{i^*} \geq \Omega(\log s)$. All other terms in the sum are non-negative, so:

$$|\Pi'| = s - \sum_{i=1}^m (k_i - 1)c_i \leq s - \Omega(\log s).$$

This establishes the claimed strict improvement. $\square$

A.10 Full Rule Set

The complete set of rules in the CERTIPROOF calculus (38 rules):

2133 **Axiom-like:** AXIOM, TRUTH.

2134 **ND Introduction:** AND_I, OR_I_L, OR_I_R, IMP_I, NOT_I, IFF_I.

2135 **ND Elimination:** AND_E_L, AND_E_R, OR_E, IMP_E (modus ponens), NOT_E, BOT_E (ex falso), DNE (double

2136 negation), IFF_E_L.

2137

2138 **Sequent Calculus:** SC_AX, SC_CUT, SC_WEAKL, SC_WEAKR, SC_AND_R, SC_IMP_R.

2139 **Resolution:** RES_UNIT, RES_FULL, RES_FACTOR.

2140 **CertiProof novel rules:** ADAPTIVE_CUT (theorem 3.7), INTERPOLANT (section 6), LEMMA_REUSE (section 6).

2141

2142

2143

2144

2145

2146

2147

2148

2149

2150

2151

2152

2153

2154

2155

2156

2157

2158

2159

2160

2161

2162

2163

2164

2165

2166

2167

2168

2169

2170

2171

2172

2173

2174

2175

2176

2177

2178

2179

2180

2181

2182

2183

2184 Manuscript submitted to ACM